

Patih

Version 3 — Complete Reference: Theory, Application, and Technical Build
A Local, Multi-Document Retrieval-Augmented Generation System
for the Indonesian Ministry of Social Affairs Regulatory Corpus
Understand the why and the how — then build it yourself

Project documentation
D:\Research\Project Data\DONE locally\patih

Document version 3: 2026-05-28

Abstract

Patih answers natural-language questions about the Indonesian Ministry of Social Affairs (*Kementerian Sosial*, *Kemensos*) regulatory corpus with traceable, article-level citations. This Version 3 document is the single complete reference. **Part I** catalogues *every* theory the system relies on — why Retrieval-Augmented Generation was chosen over the alternatives, and how each retrieval, embedding, quantisation, parsing, generation, gateway, and hallucination-defence idea works mathematically — and, crucially, each entry ends with an “*In Patih*” note stating *how the theory is operationalised in code, with concrete parameters*. **Part II** is the technical documentation: the seven-layer architecture and why it is shaped that way; the decision behind every tool; the data schemas and the full configuration/parameter reference; a component-by-component walkthrough naming each module and explaining its behaviour without reproducing code; the exact online and offline data flows; the validation methodology and the real evaluation trajectory; the evidence used to judge readiness; and a chronological build-from-scratch guide. It also records the parts of the project that are not source code: the system’s own *history* and the local-first pivot; the *free-tier economics* that shaped every provider choice, and the reason the evaluation judges could not run for free; and the *privacy model*, *secret handling*, and *operational runbook*. These are followed by trade-offs, roadmap, repository layout, and an annotated bibliography in which every source is tied to the decision it shaped. A reader who finishes both Parts should understand the system completely and be able to rebuild it.

Contents

I	Theory and Its Application	4
1	Glossary — Domain, Legal, and Technical Vocabulary	4
1.1	Indonesian regulatory and legal vocabulary	4
1.2	Machine-learning and information-retrieval vocabulary	5
1.3	Acronyms specific to the pipeline observation	6
2	The Problem and Why Retrieval-Augmented Generation	6
2.1	The task and why it is hard	6
2.2	Three ways to make an LLM answer domain questions	7
2.3	Why RAG wins here	7
2.4	The guiding principle: retrieval is the ceiling	7

3 Which Kind of RAG? The RAG Family and Patih’s Exact Choice	7
3.1 The RAG family, and what Patih uses	8
3.2 Patih’s exact configuration, in one sentence	8
3.3 Why this exact point, and not the neighbours	8
4 Representing Meaning: Embeddings	9
5 Running a Big Model on a Small Machine: Quantisation and ONNX	10
6 Finding Similar Vectors Fast: Vector Databases and HNSW	10
7 Keyword Matching: Sparse Retrieval and BM25	11
8 Combining Both: Hybrid Retrieval and Reciprocal Rank Fusion	12
9 Precise Ranking, Complete Context: Parent-Document Retrieval	12
10 Chunking Strategy	13
11 Parsing Indonesian Legislation Correctly	13
12 Chunking Documents That Have No Articles	14
13 Reading Scanned Pages: Optical Character Recognition	14
14 Positional Bias, Document Mismatch, and Summary Augmentation	14
15 The Generator: LLMs, Tokens, and Prompt Discipline	14
16 Surviving Free Tiers: Gateway, Fallback, and Rate Limiting	15
17 Two Languages: Cross-Lingual Handling	16
18 Citation Generation Theory	16
19 Hallucination Defence in Depth	16
20 Answering Across Many Laws: Multi-Document Retrieval and Routing	17
21 Why No Agentic Loop in Phase 1	17
22 Multi-Document Legal Reasoning: GraphRAG (Future)	17
23 Evaluation Theory and Metrics	18
24 Knowing When to Stay Silent: Confidence Calibration	19
25 Production Engineering Principles	19
26 Performance: Cold versus Warm Latency	19
27 Mathematical Reference	19
27.1 Symbols and notation	20
27.2 Equations	20

II	Technical Documentation and Build Guide	22
28	Project History: From a Single Document to Local-First	22
28.1	Three phases	22
28.2	Why the pivot mattered beyond hosting	22
28.3	What survived, in case public hosting returns	23
29	System Overview and the Seven-Layer Architecture	23
29.1	The chain a request actually triggers	23
30	Why the Architecture Is Shaped This Way	23
31	Technology and Tool Decisions	24
32	Complete Technology Stack	25
33	Data Schemas	27
34	Configuration and Parameter Reference	27
35	Component Catalogue — Every Module and How It Works	28
35.1	Layer 1 — Ingestion (<code>app/ingest/</code>)	28
35.2	Layer 2 — Retrieval (<code>app/retrieval/</code>)	28
35.3	Layer 3 — LLM Gateway and Generation (<code>app/llm/</code>)	29
35.4	Layer 4 — Validators (<code>app/validators/</code>)	29
35.5	Layer 5 — User Interface (<code>app/ui/</code>)	30
35.6	Layer 6 — Evaluation (<code>app/eval/</code>)	30
35.7	Layer 7 — Tooling and Observability	30
36	Code-to-Mathematics Map — Which Equation Each Module Implements	31
37	The Online Query Flow, Step by Step	32
38	The Offline Ingestion Flow, Step by Step	33
39	Validation and Evaluation in Practice	33
39.1	Per-answer validation (always on)	33
39.2	Batch evaluation (offline, labelled set)	33
40	Evaluation Trajectory (v1–v6)	33
41	Free-Tier Economics and the Free-Judge Problem	34
41.1	The real limits versus the assumed ones	34
41.2	The free-judge problem	35
41.3	The decision, and the migration trigger	35
42	How We Know the App Is Ready	36
43	Build From Scratch — Chronological Steps	36
44	Trade-offs Explicitly Accepted	37

45 Roadmap and Open Decisions	37
45.1 Phase 2 — breadth and structure	37
45.2 Phase 3 — agentic and production HITL	37
45.3 Open decisions	37
46 Repository Layout and Operations	38
46.1 Where things live	38
46.2 Run it	38
46.3 Add a document	38
47 Privacy, Security, and the Operational Runbook	38
47.1 The privacy model: what is local, what leaves the machine	39
47.2 Secret handling	39
47.3 Operational runbook: failure modes and recovery	39
48 Current Corpus Inventory (2026-05-28)	40
49 Disclaimer	41
50 Annotated Bibliography	42
50.1 Retrieval architecture	42
50.2 Embedder and language-model selection	42
50.3 Hallucination defence and citation discipline	42
50.4 Agentic and GraphRAG (Phase 2 / 3)	43
50.5 Evaluation, observability, production, cross-lingual	43
50.6 Background and adjacent (consulted, not load-bearing today)	44

Part I

Theory and Its Application

1. Glossary — Domain, Legal, and Technical Vocabulary

Patih spans three vocabularies: Indonesian legal terminology, the regulatory domain, and machine-learning / information-retrieval jargon. Everything used later is defined here.

1.1 Indonesian regulatory and legal vocabulary

Kemensos *Kementerian Sosial* — the Ministry of Social Affairs.

Permensos *Peraturan Menteri Sosial* — a ministerial regulation.

UU *Undang-Undang* — a statute / Act of Parliament (highest tier).

PP *Peraturan Pemerintah* — a Government Regulation, below an UU.

Perpres / Perbup
Presidential / Regent (*Bupati*) Regulation.

Permenkes *Peraturan Menteri Kesehatan* — Ministry-of-Health regulation.

JDIH *Jaringan Dokumentasi dan Informasi Hukum* — the national legal documentation network that publishes regulations.

BAB	Chapter (e.g. <i>BAB I — Ketentuan Umum</i>).
Bagian	Part / section of a chapter.
Pasal	Article — the canonical citeable unit of an Indonesian regulation.
Ayat	Paragraph of an article, typeset “(1), (2), ...”.
Huruf	Lettered sub-item of an ayat (“a., b., c., ...”).
Angka	Numbered sub-item, usually inside the definitions article.
Ketentuan Umum	“General Provisions” — the opening chapter defining every term used; almost always Pasal 1.
Penjelasan	The <i>Elucidation</i> appended to a law, repeating each article as commentary (a parsing hazard; see §11).
Lampiran	An attachment to a regulation (tables, forms, standards).
TPPO	<i>Tindak Pidana Perdagangan Orang</i> — the crime of human trafficking.
PMI Bermasalah	<i>Pekerja Migran Indonesia Bermasalah</i> — Indonesian migrant workers in distress.
RPJMN	<i>Rencana Pembangunan Jangka Menengah Nasional</i> — the National Mid-Term Development Plan.
SOP	Standard Operating Procedure — a non-article procedural document.
LKS	<i>Lembaga Kesejahteraan Sosial</i> — Social Welfare Institution.

1.2 Machine-learning and information-retrieval vocabulary

LLM	Large Language Model — a neural network that generates language; here, the component that writes answers.
RAG	Retrieval-Augmented Generation — ground the LLM on retrieved passages instead of parametric memory.
Token	The model’s word-piece unit (≈ 4 characters per token for Indonesian); context limits and pricing are in tokens.
Context window	Maximum tokens the LLM can consider at once.
Temperature	Randomness of generation; Patih uses 0.1 (near-greedy) for factual consistency.
Embedding	A dense vector (here $\mathbb{R}^{1024}$) encoding meaning; similar meanings lie close.
Mean pooling / L2 normalisation	Combine per-token vectors into one fixed-length unit vector.
Cosine similarity	Direction-similarity of two vectors; equals the dot product for unit vectors.
Dense / sparse retrieval	Retrieval by embedding similarity / by weighted keyword overlap.

BM25 *Best Matching 25* — the classic sparse ranker.

Hybrid retrieval

Combine dense and sparse rankers.

RRF Reciprocal Rank Fusion — merge ranked lists by summing $1/(k + \text{rank})$.

Vector database / Chroma

A store optimised for nearest-neighbour search over embeddings; Patih uses Chroma.

ANN / HNSW

Approximate Nearest Neighbour; HNSW is the graph index that finds neighbours in $\sim O(\log N)$.

Parent-document retrieval

Rank small *child* chunks but feed the larger *parent* to the LLM.

Chunk A retrievable piece of a document (a child or a parent).

ONNX Open Neural Network Exchange — a portable runtime for CPU inference.

Quantisation (FP32 / INT8)

Lower numeric precision to shrink/speed a model; FP32 = 32-bit float, INT8 = 8-bit integer.

OCR Optical Character Recognition — reading scanned page-images as text (Tesseract here).

Hallucination

A fluent but factually wrong output; the central risk.

HalluGraph / EG / RP

A KG-aligned hallucination detector with Entity-Grounding and Relation-Preservation scorers.

HITL Human-in-the-loop — routing low-confidence answers to a person.

RAGAS An LLM-as-judge evaluation framework (Faithfulness, Answer Relevancy, Context Precision/Recall).

Calibrated abstention

Refraining from answering when evidence is absent, measured as a first-class metric.

Self-RAG / CRAG / Agentic RAG

Multi-step reflective/corrective/agentic RAG patterns (deferred to later phases).

GraphRAG

Knowledge-graph-driven RAG for global/thematic questions.

LiteLLM An OpenAI-compatible router fronting many providers with fallback.

Chainlit / FastAPI / ASGI

Python chat-UI framework / API framework / the async server interface both use.

SQLite / Pickle / JSON / Regex / AST / SHA-256

Single-file DB / Python object serialisation / structured text / pattern language / parsed hierarchy / content hash.

1.3 Acronyms specific to the pipeline observation

You will see the runtime chain written as $HTTP \rightarrow \textit{sidecar} \rightarrow \textit{generator} \rightarrow \textit{retrieval} \rightarrow \textit{e5-large} \rightarrow \textit{LLM} \rightarrow \textit{validation} \rightarrow \textit{citation}$. Each token is a stage: the HTTP request reaches the FastAPI *sidecar*, which calls the *generator*, which calls *retrieval*, which embeds the query with *e5-large* and runs hybrid search; the context goes to the *LLM*; the answer is *validated*; and *citations* are extracted and rendered (detailed in §37).

2. The Problem and Why Retrieval-Augmented Generation

2.1 The task and why it is hard

Patih must answer questions about Indonesian social-welfare regulation such that *every factual claim cites the exact article it came from* (e.g. “(Pasal 5 ayat (2) huruf b)”). The domain — trafficking, problematic migrant workers, welfare administration — is high-stakes; a confident wrong answer can misdirect a real legal action. Empirically this risk is not hypothetical: a Stanford study of *commercial* legal-research tools found hallucination rates of 17–34% even in products marketed as reliable. The design therefore optimises for *verifiability*, and treats “never knowingly answer without evidence” as a hard requirement.

2.2 Three ways to make an LLM answer domain questions

Parametric memory

Ask the LLM directly. *Rejected*: it hallucinates, and niche Indonesian ministerial regulations are not reliably in its training data; it also cannot cite.

Long-context stuffing

Paste the whole regulation into every prompt. *Rejected*: token-wasteful, scales badly to a growing corpus, and suffers *Lost-in-the-Middle* (§14) — recall sags for facts placed mid-context.

Fine-tuning

Further-train a model on the regulations. *Rejected*: needs GPUs and labelled data, must be redone on every regulation change, and bakes text into opaque weights so it still cannot *cite* a source.

2.3 Why RAG wins here

RAG grounds the LLM on text retrieved at query time, so it summarises evidence rather than recalling from memory. This delivers verifiability (each claim points back to an article and page), freshness (adding a regulation is indexing, not retraining — vital for a corpus that grows), low cost (no training; free-tier inference on a CPU laptop), and hallucination control (refuse when evidence is absent). The closed-book/open-book analogy is exact: a bare LLM answers from memory; RAG opens the book and quotes the page.

2.4 The guiding principle: retrieval is the ceiling

A RAG answer can be no better than the passages retrieved: if the right article is not retrieved, no LLM cleverness produces a correct cited answer. Benchmarks confirm that swapping the embedder moves correctness far more than swapping the LLM, and that many “hallucinations” are really retrieval failures. Consequently Patih spends most of its engineering on *retrieval quality* — hybrid search, parent-document expansion, careful chunking, correct parsing — and treats the LLM as a constrained summariser that is validated after the fact.

In Patih. This principle is why Part II’s complexity lives in `app/retrieval/` and `app/ingest/`, while `app/llm/` is thin: detect language, build a strict prompt, call a provider, return.

3. Which Kind of RAG? The RAG Family and Patih’s Exact Choice

“RAG” is not one architecture but a *family* of variants that differ along four independent axes: **(a) how retrieval is done** (dense / sparse / hybrid / late interaction), **(b) the unit of retrieval** (flat chunk / parent-document / tree / graph), **(c) the control flow** (single-pass / reflective / corrective / agentic), and **(d) what augments the query or context** (none / query expansion / HyDE / summary). Choosing “RAG” (§2) only narrows the design to this family; this section pins the *exact point* Patih occupies and why, and lists the variants deliberately not used.

3.1 The RAG family, and what Patih uses

RAG variant	What it is	In Patih?
Naive / Standard RAG	Single dense top- <i>k</i> retrieve → generate.	Baseline only; superseded.
Hybrid RAG	Dense + sparse retrieval fused (here by RRF).	Yes (axis a).
Parent-Document RAG	Rank small children, feed the larger parent to the LLM.	Yes (axis b).
Reranking RAG	A cross-encoder re-scores the top- <i>k</i> (query,passage) pairs.	No — paid/GPU; RRF + parent expansion suffices.
Multi-vector / late interaction (ColBERT)	Token-level vectors, finer matching.	No — heavier index, unneeded at this scale.
HyDE	Embed a <i>hypothetical</i> answer, then retrieve.	No — adds an LLM call per query.
Multi-query / query expansion	Generate query variants and union results.	No — extra LLM calls; free-tier cost.
RAPTOR / tree RAG	Build a recursive summary tree over chunks.	No — Patih instead uses the <i>natural</i> legal hierarchy (BAB/Pasal/ayat) for the same benefit.
Adaptive RAG	A classifier routes by query complexity.	Not yet — Phase 2.
Self-RAG	Reflective retrieve/critique tokens decide when/what to retrieve.	No — Phase 3.
Corrective RAG (CRAG)	A retrieval-evaluator gates a fallback search.	No — Phase 3.
Agentic RAG	Multi-step retrieve-reason loop (a POMDP).	No — Phase 3 (cost, safety; §21).
GraphRAG	Knowledge-graph + community detection + map-reduce QA.	No — Phase 2 (§22).
Long-context “RAG-less”	Stuff the whole document; skip retrieval.	No — Lost-in-the-Middle, cost (§2).

3.2 Patih’s exact configuration, in one sentence

Patih is a **single-pass, Hybrid (dense + sparse, RRF-fused) Parent-Document RAG**, augmented with two domain-specific retrieval steps — *cross-reference resolution* and an *always-on*

definitions article — and **gated by a post-hoc citation-and-grounding validation layer** (closest in spirit to *citation-enforced RAG*). In taxonomy terms: *advanced on the retrieval axes (a, b), deliberately simple on the control-flow axis (c), and lightly augmented on axis (d)*.

3.3 Why this exact point, and not the neighbours

Hybrid, not dense-only.

Legal questions need both meaning (“bentuk eksploitasi” $\approx$ “a. pelacuran ...”) and exact-token matching (the literal “Pasal 5”). Dense alone blurs article numbers; sparse alone misses paraphrase. Hence dense + sparse fused (§8).

Parent-document, not flat chunks.

The *citeable* unit (the whole *Pasal*) differs from the *best ranking* unit (a single *ayat/huruf*). Parent-document retrieval keeps both: sharp ranking, complete citable context (§9).

Single-pass, not agentic/self/corrective.

Reflective and agentic loops multiply LLM calls (untenable on free tiers), add failure modes, and are harder to validate. A single retrieve-then-generate pass is cheaper, predictable, and easy to gate with validators — the right Phase 1 trade-off (§21).

Hierarchy, not RAPTOR trees.

Indonesian law already provides an authoritative hierarchy (BAB/Bagian/Pasal/ayat/huruf); Patih parses and chunks along it rather than building a synthetic summary tree — the natural structure gives the granularity RAPTOR engineers artificially.

No reranker / HyDE / multi-query.

Each adds either a paid cross-encoder/GPU or an extra LLM call per query. RRF plus parent expansion already meets the recall target, so the added cost is not justified in Phase 1.

Not GraphRAG yet.

Phase 1 is anchored on a single regulation, so cross-regulation graph reasoning has nothing to reason *across*. GraphRAG becomes valuable once the multi-document corpus needs thematic “how do UU X, PP Y, Permensos Z interact” answers — a Phase 2 item (§22).

Validation layer on top.

Because the domain is legal, the answer is not trusted on its face: it is gated by citation existence and HalluGraph grounding (§19). This is the feature that makes Patih a *citation-enforced RAG* rather than a plain one.

The result is a system that is as sophisticated as it needs to be on the axes that drive correctness (retrieval, chunking, validation) and deliberately minimal on the axis that drives cost and risk (control flow). Phase 2/3 move along the control-flow and structure axes (Adaptive $\rightarrow$ GraphRAG $\rightarrow$ Agentic) as the corpus and question complexity grow.

4. Representing Meaning: Embeddings

Theory. An embedding model maps a passage to a fixed-length vector whose geometry encodes meaning; a query and a relevant passage are trained to land close under cosine similarity. The transformer emits one vector per token; these are *mean-pooled* and *L2-normalised*, so cosine similarity reduces to a dot product. Indonesian is comparatively low-resource, so a *multilingual* encoder is essential.

Deeper theory: from tokens to a meaning vector. Three layers of theory sit under a single embedding call. (i) *Tokenisation.* Text is first split into subword *tokens* by a byte-pair-encoding-style vocabulary: frequent words map to one token, rare words split into pieces. This bounds the

vocabulary yet can represent any string; for Indonesian it averages ~ 4 characters per token. (ii) *The transformer and self-attention*. Each token becomes a vector, and *self-attention* lets every token’s representation be a learned weighted blend of all others: with queries Q , keys K , values V (linear projections of the token vectors), attention is $\text{softmax}(QK^\top/\sqrt{d_k})V$. Stacking many such layers yields a *contextual* vector per token — the same word gets different vectors in different sentences, so “korban” in “korban TPPO” embeds differently from “korban” in a traffic-accident sentence. (iii) *Contrastive training (InfoNCE)*. e5 is trained so that a query and its true passage (a positive) are pulled together while unrelated passages (negatives) are pushed apart, minimising $-\log \frac{e^{\text{sim}(q,c^+)/\tau}}{\sum_c e^{\text{sim}(q,c)/\tau}}$. This is precisely why a query lands close to a *relevant* passage rather than a merely lexically similar one, and why the **query:/passage:** prefixes matter — they tell the model which role each text plays. *Mean* pooling is preferred over single-token (CLS) pooling for retrieval robustness; L2-normalisation (E2) then places every vector on the unit sphere, so cosine becomes a dot product (E3–E4).

In Patih. The embedder is **multilingual-e5-large**, vector dimension $d = 1024$, run on CPU through ONNX (`app/retrieval/embedder.py`). It supports three backends — a cloud embedding API, the local ONNX model, and a small English-only model used only for development — pinned by the `EMBEDDER_BACKEND` environment variable so the index and the query path always share a dimension (the local install pins `onnx`). The e5 prefix convention is honoured: passages carry **"passage: "** (baked into each chunk’s `text_for_embed` at ingest), queries get **"query: "** at search time. Tokenisation truncates at 512 tokens; batch size at indexing is 16. An English-only fallback (MiniLM, $d = 384$) measurably underperformed on Indonesian legal text, which is why the multilingual model is mandatory.

Worked example. The chunk for Pasal 5 ayat (2) is stored with embed text **"passage: [Permensos 8/2023 -- Pasal 5] ...bentuk eksploitasi ..."** and encoded once at ingest into a 1024-dimensional unit vector. At query time **"apa saja bentuk eksploitasi?"** becomes **"query: apa saja bentuk eksploitasi?"**, is encoded to its own unit vector, and Chroma returns the chunks whose vectors have the largest dot product with it (E3–E4). Because e5 was trained on the query/passage roles, the definitional ayat ranks above an incidental mention of “eksploitasi” elsewhere *even though the surface words differ* — the query says “bentuk”, the article enumerates “a. pelacuran; b. kerja paksa; ...”. This is the concrete payoff of *dense* retrieval: matching by meaning, not by shared words.

5. Running a Big Model on a Small Machine: Quantisation and ONNX

Theory. Quantisation stores a model’s weights at lower precision. INT8 maps a float range to 8-bit integers via a scale s and zero-point z :

$$s = \frac{W_{\max} - W_{\min}}{255}, \quad z = \text{round}(-W_{\min}/s), \quad W_{ij}^{\text{int8}} = \text{clip}(\text{round}(W_{ij}/s) + z, -128, 127).$$

For retrieval the accuracy loss is under $\sim 1\%$ nDCG@10 while size drops $\sim 4\times$ and CPU throughput roughly doubles. The ONNX runtime executes the model portably on CPU (no GPU).

In Patih. `deploy/scripts/quantize_e5.py` exports e5-large to ONNX and attempts INT8 quantisation, with a full-precision (FP32) fallback. The INT8 path failed on Windows: the quantiser printed a Unicode arrow to a legacy-code-page console, raising a `charmap` encoding error, so the system fell back to FP32 (~ 2.3 GB on disk, ~ 2 GB RAM). FP32 is correct but slower — the direct cause of the slow first query (§26). The recorded fix is to retry with `PYTHONUTF8=1` set so the quantiser’s console output does not crash.

6. Finding Similar Vectors Fast: Vector Databases and HNSW

Theory. A naïve nearest-neighbour search compares the query against every stored vector ($O(N)$). An HNSW (Hierarchical Navigable Small World) index instead builds a layered proximity graph navigated coarse-to-fine, returning approximate neighbours in $\sim O(\log N)$. Cosine distance pairs naturally with L2-normalised embeddings.

Deeper theory: why approximate, and why a small-world graph. In high dimensions the *curse of dimensionality* bites: pairwise distances concentrate (almost everything looks equidistant) and tree indices such as k-d trees degrade to near-linear scans. Practical systems therefore accept *approximate* nearest neighbours, trading a small recall loss for a large speed gain. HNSW builds a hierarchy of *navigable small-world* graphs — each node links to a few near neighbours plus occasional long-range links — so a *greedy* walk (always step to the neighbour closest to the query) reaches the true neighbourhood in $\sim O(\log N)$ hops. The sparse upper layers route quickly across the space; the dense bottom layer refines locally. The recall/latency trade is explicitly tunable: more links per node and a wider search beam raise recall at the cost of speed. At Patih’s $\sim 1,800$ vectors an exact scan is also instant, so HNSW here is insurance for corpus growth rather than a present necessity — but it is the reason the design scales without an architectural change.

In Patih. The vector store is Chroma, wrapped by `app/retrieval/vector_store.py`, configured for the cosine space. It persists to `data/chroma/`. Chroma metadata values must be scalar, so list fields (tags, cross-references) are JSON-encoded on write and decoded on read. At the present scale ($\sim 1,800$ vectors) exact search would also be fast; HNSW matters as the corpus grows.

7. Keyword Matching: Sparse Retrieval and BM25

Theory. BM25 ranks a document D for query Q :

$$\text{BM25}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \frac{f(t, D)(k_1 + 1)}{f(t, D) + k_1(1 - b + b|D|/\text{avgdl})}, \quad \text{IDF}(t) = \log \frac{N - n(t) + 0.5}{n(t) + 0.5}.$$

It rewards rare terms (IDF), saturates repeated terms (k_1), and normalises for length (b , avgdl). Sparse retrieval catches *exact* matches dense embeddings blur — above all literal article numbers like “Pasal 5”.

Deeper theory: from the vector space model to the Probabilistic Relevance Framework. BM25 is the mature descendant of a long lineage. The *vector space model* represented documents as term-weight vectors and ranked by cosine; *TF-IDF* weighting (term frequency $\times$ inverse document frequency) formalised the intuition “terms frequent *here* but rare *overall* are informative”. BM25 instead derives from the *Probabilistic Relevance Framework*, which ranks documents by the estimated odds of relevance given the query terms. Two empirical corrections give BM25 its characteristic shape. First, term frequency is passed through a *saturating* function $\frac{f(k_1+1)}{f+k_1(\cdot)}$, so the tenth occurrence of a word adds far less than the first — real relevance saturates. Second, a *length normalisation* controlled by $b \in [0, 1]$ discounts long documents that accumulate matches by sheer size. The $+0.5$ smoothing terms in IDF (E8) are a Bayesian prior that keeps the logarithm well-behaved for extremely rare or extremely common terms. k_1 (typically 1.2–2.0) sets how quickly saturation bites. Patih keeps the defaults because its chunks are short and uniform (a single ayat or huruf), so length normalisation barely matters — what matters is the exact-term boost that recovers a literal “Pasal N”.

In Patih. `app/retrieval/bm25_store.py` builds a BM25Okapi index over all child chunks, tokenising by lowercasing and stripping punctuation *with no stemming* (the Indonesian stemmer Sastrawi over-reduces legal terms). The index is pickled to `data/bm25/`. The embed text it

indexes keeps the document label and article references, helping queries that mention a specific Pasal number.

Worked example. For “*sanksi Pasal 5*” the rare tokens “pasal” and “5” carry high IDF (E8), so the actual Pasal 5 chunk outranks chunks that merely discuss sanctions — something dense search can miss, because it treats “Pasal 5” as a weak numeric token. The no-stemming choice is concrete: Sastrawi reduces both “perdagangan” (trade/trafficking) and “pedagang” (merchant) to the root “dagang”, which would blur “perdagangan orang” (human trafficking) toward an unrelated word; Patih keeps surface forms so the legal term stays distinct in the index. This is the complementary half of retrieval: dense search generalises meaning, sparse search nails the exact token — which is why both are fused (§8).

8. Combining Both: Hybrid Retrieval and Reciprocal Rank Fusion

Theory. Dense and sparse retrieval have complementary strengths; RRF merges their ranked lists using *ranks only*, so the incomparable score scales never need calibration. An item at rank r in list L_j contributes $1/(k + r)$, summed across lists, with $k = 60$:

$$\text{RRF}(d) = \sum_j \frac{1}{k + \text{rank}_{L_j}(d)}.$$

Deeper theory: rank aggregation. Merging ranked lists is a *rank-aggregation* problem with a long history — the Condorcet method elects the candidate that wins pairwise majorities. *Score-based* fusion (CombSUM, CombMNZ) adds the rankers’ raw scores, but BM25 scores and cosine similarities live on incomparable scales, so they must first be normalised, and normalisation is brittle (a single outlier score distorts the blend). RRF avoids this by discarding scores and using only *ranks*: each contribution $1/(k + r)$ is bounded in $[0, 1/(k + 1)]$ regardless of the underlying scorer, so neither ranker can dominate by being numerically loud. The constant $k=60$ deliberately compresses the gap between top ranks (rank 1 vs rank 3), so *agreement across* lists outweighs *extreme confidence within* one list. RRF was shown to beat both Condorcet and learning-to-rank baselines while requiring no training and no tuning — exactly the robustness a zero-budget system wants.

In Patih. `app/retrieval/hybrid.py` fuses the dense list (top `top_k_dense` = 15) and sparse list (top `top_k_sparse` = 15) into the top `top_k_fused` (default 8; tuned to 12 during evaluation). $k = 60$ is the original RRF default. No normalisation is performed because only ranks are used.

Worked example. Suppose chunk A is dense-rank 1 but sparse-rank 9, while chunk B is dense-rank 3 and sparse-rank 2. With $k = 60$ (E9): $\text{RRF}(A) = \frac{1}{61} + \frac{1}{69} = 0.0309$ and $\text{RRF}(B) = \frac{1}{63} + \frac{1}{62} = 0.0320$, so B fuses *above* A : a chunk both rankers like moderately beats one only the dense ranker loves. The large constant $k = 60$ flattens the gap between the very top ranks, so no single list can dominate the fusion — exactly the robustness that lets Patih combine two scoring scales it never has to calibrate against each other.

9. Precise Ranking, Complete Context: Parent-Document Retrieval

Theory. Embedding a long article yields a blurry average vector that ranks poorly; feeding the LLM a tiny fragment starves it of context. Parent-document retrieval ranks on small *children* but answers with their *parents*.

In Patih. `app/retrieval/parent_expander.py` maps each fused child to its parent via the child’s `parent_id`, de-duplicates so each parent appears once, and keeps the best child score as

the parent’s score. For regulations the parent is the *Pasal* and children are its *ayat/huruf*; for reference documents the parent is a section and children are size-windows (§12). A parent-lookup table (`data/bm25/parent_lookup.json`) and a merged child-lookup across all parsed documents make this resolution work corpus-wide.

Worked example. A query about exploitation matches the child `permensos-8-2023::pasal5::ayat2::huruf` (the single line “a. pelacuran”). Ranking on that precise child is sharp, but answering from it alone would omit the other twelve forms. The expander follows the child’s `parent_id` to `permensos-8-2023::pasal5` and hands the LLM the *whole* Pasal 5 — forms a–m — so the answer can enumerate all of them under a single citation “(Pasal 5 ayat (2))”. Small unit to rank, large unit to read: that is the entire point of the pattern.

10. Chunking Strategy

Theory. Chunk size trades ranking precision (small is better) against context completeness (large is better) and interacts with Lost-in-the-Middle (short final contexts are recalled better). Two extra retrieval-time augmentations help legal text: an *always-on definitions article* (Pasal 1, where every term is defined) and *cross-reference resolution* (pulling in articles a retrieved article explicitly points to).

In Patih. `app/ingest/chunker.py` produces parents (*Pasal*) and children (*ayat/huruf*) with stable identifiers (`doc_id::pasalN::ayatM::hurufX`), a `parent_id`, a `source_page`, the pre-fixed `text_for_embed`, and an `always_on` tag on Pasal 1. At retrieval, `always_on.py` prepends the definitions article and `cross_ref_resolver.py` adds up to 3 referenced articles (§20 explains the per-document variants).

11. Parsing Indonesian Legislation Correctly

Theory. Regulations nest as $BAB \rightarrow Bagian \rightarrow Pasal \rightarrow ayat \rightarrow huruf$. A robust parser must survive real-world formatting. Five hazards surfaced when moving from one document to many, each fixed:

Penjelasan inflation

The *Penjelasan* repeats every article as commentary, doubling the count. Fix: truncate at the *Penjelasan* heading, but only past 40% of the document so a stray early mention does not delete the body. (UU 39/1999: a mis-parsed 209 articles $\rightarrow$ the true 106.)

Omnibus laws

An omnibus that amends many laws (UU 6/2023, *Cipta Kerja*) embeds dozens of other laws, yielding ~ 700 spurious articles numbered up to 4024. Fix: exclude it from the corpus.

Pre-BAB articles

Short regulations place articles *before* the first BAB (BAB headings live only in a trailing attachment). Fix: also scan the pre-BAB region. (Permensos 22/2016: 2 $\rightarrow$ 4; a civil-service regulation: 4 $\rightarrow$ 20.)

Trailing-period headers

Older laws write “Pasal 5.”; the article pattern tolerates an optional period. (Restores UU 9/1961.)

Cross-chapter duplicates

Article numbers are unique within a law; duplicates across chapters are artefacts. Fix: keep the first, drop the rest.

In Patih. All five live in `app/ingest/structure_parser.py`, which builds the AST with regular expressions and returns a flag of zero parsed articles to signal “not a regulation” (routing the document to the generic ingester instead).

Worked example (Penjelasan). UU 39/1999 prints “Pasal 1 ... Pasal 106” once in the *Batang Tubuh* (body) and again inside the *Penjelasan* (elucidation). A naive article scan therefore counts 209. The parser finds the “PENJELASAN” heading at ~65% of the text, truncates there, and re-counts: 106 — the true figure. The 40% position guard stops a document that merely *mentions* “penjelasan” early (e.g. in its *Mengingat* preamble) from being cut in half. The *pre-BAB* fix is the mirror case: Permensos 22/2016 places Pasal 1–4 *before* “BAB I”, which only appears inside its *Lampiran*; scanning the pre-BAB region recovers those four articles instead of dropping them.

12. Chunking Documents That Have No Articles

Theory. When a document has no *Pasal* (SOPs, statistics, the RPJMN summary), chunk it by heading and size. Detect heading-like lines (short all-capitals, or numbered headings); each starts a new *section* (parent); the body is cut into overlapping windows (children) so a sentence split across a boundary still appears whole in one window.

In Patih. `app/ingest/generic_ingest.py` implements this with child windows of ~1,200 characters and ~150-character overlap, falling back to one section per page when headings are weak. It emits the *same* parsed-JSON schema as regulations but with `pasal=null` and a `section_title`; downstream the absent article switches the citation style to “document name + page” (§15, §37).

13. Reading Scanned Pages: Optical Character Recognition

Theory. A scanned PDF has no text layer, only page-images; OCR converts the image to text. Rendering at higher resolution improves accuracy; OCR text is noisier than native text (hence the period-tolerant parser above).

In Patih. `app/ingest/pdf_loader.py` first tries native extraction (PyMuPDF) and assesses quality (characters-per-page, non-ASCII ratio). If quality is poor or OCR is forced, it renders each page at 2× and runs Tesseract with the Indonesian model (`ind`). A helper auto-locates the Tesseract binary and points it at the project-local language data (`models/tessdata/`), so OCR works unattended, including from the folder watcher. (UU 9/1961, a pure scan, was recovered to 9 articles this way.)

14. Positional Bias, Document Mismatch, and Summary Augmentation

Lost-in-the-Middle (Liu et al., 2023)

Long-context LLMs recall the start and end of context far better than the middle (a U-shaped curve). *Implication:* keep contexts short, most-relevant article early — which Patih’s small-chunk, parent-expansion design produces.

DRM — Document-Level Retrieval Mismatch (Reuter et al., 2025)

In a corpus of many similar documents, the retriever may pull from the *wrong* source. This is the central multi-document risk; Patih answers it with per-document cross-reference resolution and routing (§20).

SAC — Summary-Augmented Chunking (Reuter et al., 2025)

Prepend a document-level summary to each chunk to re-inject lost global context; generic summaries surprisingly beat expert ones. Patih uses a light label (document name in each chunk’s embed text); full SAC is a roadmap item.

15. The Generator: LLMs, Tokens, and Prompt Discipline

Theory. An LLM reads/writes *tokens* within a *context window*; *temperature* controls randomness; the *system prompt* sets the rules and the *user prompt* carries context and question; *few-shot* examples demonstrate format. Citation discipline is achieved by instruction, not training. Near-greedy decoding is the temperature softmax with low T : $P_T(t_i | t_{<i}) = e^{z_i/T} / \sum_j e^{z_j/T}$, $T = 0.1$, which favours verbatim quotation over paraphrase.

Deeper theory: how an LLM decodes, and why low temperature. An LLM is trained to minimise *cross-entropy* on next-token prediction — maximising the probability of the actually-observed next token — which is equivalent to minimising *perplexity* (its average “surprise”). At inference the final layer emits a *logit* z_i per vocabulary token; the softmax turns logits into a probability distribution. *Decoding* then selects tokens: *greedy* takes the argmax; *beam search* keeps several partial sequences; *top-k* and *top-p* (*nucleus*) sampling draw from only the most probable tokens; *temperature* T rescales logits (z_i/T) before the softmax — $T \rightarrow 0$ approaches greedy (sharp, deterministic), $T > 1$ flattens the distribution (diverse, creative). Patih uses $T = 0.1$: near-deterministic, so the model reproduces the source wording and a fixed citation format rather than paraphrasing. The same training objective also explains *why hallucination is intrinsic*: the model is optimised to produce *plausible* continuations, not *true* ones — which is the theoretical reason an independent grounding check (§19) is mandatory, not optional.

In Patih. `app/llm/prompts.py` loads the system prompt (`configs/prompts/system_id.md`) and builds the messages. The prompt mandates: answer only from context; always cite the article in the exact format “(Pasal X ayat (Y) huruf z)”; *name the source regulation in prose* because the context can mix several laws; refuse if the answer is absent; never invent article or regulation numbers; for reference documents cite by name and page instead of by article. Each context chunk is labelled per document — “[Pasal N — BAB X — *label*]” or “[*section title* — hal. P — *label*]” — so the model attributes each fact correctly. Decoding uses $T = 0.1$, `max_tokens = 1500`. A bilingual prompt and few-shot examples exist for English queries.

Worked example. At $T = 0.1$ the softmax (E11) sharpens toward the single most-probable next token, so when the context line reads “a. pelacuran,” the model reproduces that exact wording instead of paraphrasing to “aktivitas terkait prostitusi”. A higher temperature would invite fluent paraphrase — precisely what a verbatim legal quotation must avoid. The per-document chunk labels are what let the model write “Menurut UU 35/2014 (Pasal 1 angka 1) ...” rather than an ambiguous “Pasal 1”, because the same article number exists in many laws at once.

16. Surviving Free Tiers: Gateway, Fallback, and Rate Limiting

Theory. Free LLM tiers cap requests per minute (RPM) and tokens per day. A *token bucket* pre-empts limits: a bucket of capacity C refills at rate r ; each request spends one token; an empty bucket waits: $b_{t+\Delta t} = \min(C, b_t + \Delta t \cdot r) - 1$. Chaining several providers aggregates capacity ($\Lambda = \sum_p \Lambda_p \approx 70\text{--}90$ RPM here) against a modest expected arrival rate ($\lambda = UQ/(60T_w) \approx 2.1$ RPM for 100 users over 8 hours), giving ample headroom.

In Patih. `app/llm/rate_limiter.py` implements a per-provider token bucket and registry (Gemini 10, Groq 30, Cerebras 30, OpenRouter 20 RPM). `app/llm/gateway.py` configures a LiteLLM router with the four providers in priority order and a fallback chain, iterating attempts so per-attempt latency and the provider actually used are recorded; retry timeouts are tuned

tight (~ 5 s/attempt, ~ 15 s budget) to avoid tail-latency cliffs. Llama 3.3 70B (Groq) is the workhorse.

Worked example. Groq’s bucket has capacity $C = 30$ and refill $r = 30/60 = 0.5$ tokens/s (E28). After a burst empties it, the next request *waits* ~ 2 s for a token instead of firing and receiving an HTTP 429. If Groq’s daily token budget is exhausted, the router falls through to Gemini, then Cerebras, then OpenRouter; the aggregate capacity $\Lambda \approx 70\text{--}90$ RPM (E24) comfortably absorbs the expected $\lambda \approx 2.1$ RPM (E25), and Little’s law (E26) bounds the queue so the P95 latency budget holds.

17. Two Languages: Cross-Lingual Handling

Theory. The corpus is Indonesian; the cheapest reliable cross-lingual recipe is “translate the query, not the corpus”. Citations must stay verbatim in Indonesian because a translated “Article 5 paragraph (2)” is not a valid legal reference.

In Patih. `app/llm/lang_detect.py` (lingua-py) detects ID/EN/mixed with a confidence threshold and Indonesian default; `app/llm/translator.py` translates an English query to Indonesian for retrieval (cloud model, SQLite-cached, graceful fallback to the original). The answer can be produced in the user’s language while article citations remain Indonesian.

18. Citation Generation Theory

Theory. Citation metrics can over-penalise *extra* citations; a model that adds a relevant nearby article should not be scored as wrong on recall. So precision and recall are reported separately, and “generate-then-refine” approaches add missing and prune irrelevant citations without changing answer text. Abstention (refusing when unsupported) is a distinct, independently-measured behaviour.

In Patih. Patih’s evaluation separates micro-recall from precision (§23); the live validators (§19) enforce that cited articles actually exist; refusal precision is a first-class metric on out-of-scope questions.

19. Hallucination Defence in Depth

Theory. No single check is trusted; independent cheap checks stack, and their combined miss-rate multiplies down: $p_L = p_0 \prod_{\ell=1}^L (1 - c_\ell)$ where c_ℓ is layer ℓ ’s catch rate. HalluGraph contributes two KG-aligned scorers — Entity-Grounding $EG = |\mathcal{E}(r) \cap \mathcal{E}(\text{ctx})|/|\mathcal{E}(r)|$ and Relation-Preservation $RP = |\mathcal{R}(r) \cap \mathcal{R}(\text{ctx})|/|\mathcal{R}(r)|$ — which separate faithful from hallucinated answers at AUC 0.979 on structured legal documents, beating plain semantic similarity (which tolerates entity substitution).

Deeper theory: a taxonomy of hallucination, and why set-grounding detects it. Hallucination is split along two axes: *intrinsic* (the output contradicts the provided context) vs. *extrinsic* (it adds claims unverifiable from the context); and *faithfulness* (entailed by the source) vs. *factuality* (true in the world). A RAG system’s contract is *faithfulness*: say only what the retrieved context supports. The gold-standard faithfulness test is *natural-language inference* (NLI) — does the context *entail* each claim? — but NLI needs its own model and is costly. HalluGraph’s insight is that on *structured* legal text, faithfulness is well-approximated by *set membership*: extract the entities and relations of the answer and check they are subsets of the context’s (E16–E17). This is cheap and, crucially, *robust to entity substitution* — a plausible-but-wrong swapped article number changes the entity set and is caught, whereas a cosine-similarity check would still rate the sentence “similar”. A detector’s quality is summarised by the *AUC* of its ROC curve (E18): the probability it scores a faithful answer above a hallucinated one. Because the four checks are *independent*, their miss-rates multiply (E19): if each catches even

70–90% of its error type, the residual after all four is small — the formal justification for “defence in depth” rather than one perfect check.

In Patih. Four validators run after generation, orchestrated by `app/validators/pipeline.py`: `citation_extractor.py` (regex pull of every “Pasal X ayat (Y) huruf z”); `whitelist_validator.py` (range check + Chroma existence; valid if found in *any* retrieved document; falls back to confirm a cited *huruf* appears in the ayat text); `entity_grounding.py` (EG over a curated legal-term set); and `relation_preservation.py` (RP per sentence, skipping footer/citation-only lines). A threshold gate raises a HITL flag if a citation is invalid, or $EG < 0.95$, or $RP < 0.85$, writes the case to `data/hitl_queue.jsonl`, and colours the confidence badge. Short definitional answers can legitimately score low on RP — a known false-positive that errs toward caution.

Worked example. Answer: “Korban TPPO berhak atas rehabilitasi sosial (Pasal 25 ayat (1))”. Citation extraction yields {Pasal 25 ayat 1}; the whitelist confirms Pasal 25 exists in the retrieved Permensos 8/2023, so it is valid. The entities {korban TPPO, rehabilitasi sosial} both appear in the retrieved context, so $EG = 1.0$ (E16); the sentence’s content words overlap the context, so $RP \geq 0.85$ (E17) — the badge is green, no HITL flag. Now suppose the model wrote “Pasal 250”: the range check fails immediately, the citation is marked invalid, and the gate (E19) raises a HITL flag *regardless* of EG/RP. The cheap exact check catches a fabricated number that a similarity score would happily tolerate — the reason the layers are independent.

20. Answering Across Many Laws: Multi-Document Retrieval and Routing

Theory. With many laws in one index, a question competes across all of them; the hybrid ranker surfaces the most relevant articles regardless of source, and the dominant retrieved document effectively wins (implicit routing, no separate classifier). The DRM risk (§14) is countered by keeping cross-references and definitions *document-scoped*.

In Patih. In `app/retrieval/pipeline.py`: cross-reference resolution uses the *same* document a passage came from (a “Pasal 5” inside a UU resolves to that UU’s Pasal 5, never another law’s); the always-on definitions article is Pasal 1 of the *dominant* retrieved document, not a hard-coded one; and an optional `doc_filter` restricts retrieval to a single law (used by single-document evaluation, applied as a Chroma metadata filter on the dense side and a chunk-id prefix filter on the BM25 side).

Worked example. “Apa definisi anak?” is asked against all 22 documents. The hybrid ranker returns mostly chunks from UU 35/2014 (Child Protection); UU 35/2014 is therefore the dominant document, so its Pasal 1 (not Permensos 8/2023’s) is prepended as the definitions article, and the answer cites “UU 35/2014 (Pasal 1 angka 1)”. Had cross-references been resolved globally instead of per document, a “Pasal 5” inside UU 35/2014 could have pulled in Permensos 8/2023’s Pasal 5 — the exact Document-Level Retrieval Mismatch the per-document scoping prevents. This was verified live: child-protection → UU 35/2014, human-rights → UU 39/1999, trafficking → Permensos 8/2023.

21. Why No Agentic Loop in Phase 1

Theory. Multi-step retrieval–reasoning agents are powerful but multiply LLM calls and add failure modes. Agentic RAG can be formalised as a finite-horizon partially observable Markov decision process $\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma, H)$ with objective $J(\pi) = \mathbb{E}_{\pi} \sum_{t=0}^H \gamma^t R(s_t, a_t)$; surveys catalogue compounding hallucination, memory poisoning, and cascading tool-execution risks. Patterns surveyed: Self-RAG (retrieve/critique tokens), CRAG (a corrective retrieval evaluator gating fallback search), Doctor-RAG (failure-aware repair), and Chain-of-Retrieval (dynamic query reformulation).

In Patih. Phase 1 uses a single retrieve-then-generate pass: cheaper under free-tier limits, more predictable, and far easier to validate. Agentic reasoning (bounded to ~ 3 hops) is a Phase 3 item.

22. Multi-Document Legal Reasoning: GraphRAG (Future)

Theory. GraphRAG builds an LLM-extracted knowledge graph of entities and typed relations, detects communities, and answers thematic/global questions by map-reduce over them — well suited to “how do UU X, PP Y, and Permensos Z interact?”. A domain-partitioned hybrid (RAG modules + a legal KG + an orchestrator) has shown large pass-rate gains over RAG-only baselines on legal reasoning.

In Patih. Deferred to Phase 2/3; Phase 1 relies on hybrid retrieval plus lightweight cross-reference resolution. A KG of Pasal–ayat–reference links is the planned next structural step.

23. Evaluation Theory and Metrics

Theory. Quality is measured, not asserted:

Citation accuracy

$P = |M \cap G|/|M|$, $R = |M \cap G|/|G|$, $F_1 = 2PR/(P+R)$, micro-recall $R_{\text{micro}} = \sum_i |M_i \cap G_i| / \sum_i |G_i|$, and mean Jaccard — M = model citations, G = ground truth, at Pasal/ayat/huruf granularity.

Refusal precision

TP/(TP + FP) on must-refuse questions — the abstention metric.

Latency percentiles

P50/P95/P99 per stage.

Faithfulness (RAGAS)

fraction of the answer’s atomic claims entailed by context.

Answer Relevancy (RAGAS)

cosine alignment between the question and questions synthesised back from the answer.

Context Precision/Recall (RAGAS)

ordering and coverage quality of retrieved chunks.

Deeper theory: information-retrieval metrics. The metrics descend from classical IR.

Precision and *recall* trade off — returning more citations raises recall but usually lowers precision — and F_1 is their *harmonic* mean, which (unlike the arithmetic mean) punishes a large imbalance between the two. *Micro*-averaging pools every question’s hits and misses before dividing (long-answer questions weigh more), whereas *macro*-averaging averages per-question scores (every question weighs equally); Patih watches micro-recall as the headline because total cited-article correctness is what matters. Ranking-aware metrics extend these: *Mean Reciprocal Rank* rewards placing the first relevant item early; *Mean Average Precision* integrates precision across recall levels; *nDCG* additionally discounts hits that appear lower in the list and normalises against the ideal ordering. RAGAS Context Precision (E22) is an nDCG-flavoured measure of whether the relevant chunks were ranked first — which matters precisely because of retrieval-is-the-ceiling (§2.4): the right article must be near the *top* of retrieval, not merely somewhere in it.

In Patih. `app/eval/` implements each: `citation_accuracy.py`, `refusal_precision.py`, `latency_meter.py`, and `ragas_runner.py` (RAGAS judge with SQLite cache and NaN filtering), driven by a tiered

50-question set in `testset_loader.py` and reported by `report.py`. Acceptance thresholds are in §39; the real run-by-run trajectory is in §40.

Worked example. A question expects $G = \{\text{Pasal 5, Pasal 6}\}$. The model answers citing $M = \{\text{Pasal 5, Pasal 6, Pasal 4}\}$. Then $P = |M \cap G|/|M| = 2/3 = 0.67$ and $R = |M \cap G|/|G| = 2/2 = 1.0$ (E12): recall is perfect, precision is dinged by the extra Pasal 4 — which may actually be relevant. Reporting P and R *separately* (E12–E13) prevents an over-citation from masquerading as a miss, and is exactly why the v5/v6 runs (§40) diagnosed the citation-accuracy gap as a recall problem on cross-reference questions, not a precision problem.

24. Knowing When to Stay Silent: Confidence Calibration

Theory. Stated confidence should match real reliability, and the system should abstain when evidence is absent. Over-promising “100% accuracy” in a legal domain is itself a safety failure; most LLMs fail abstention even when instructed.

In Patih. Patih targets $\geq 90\%$ citation accuracy with a citation on every answer and human review of flagged cases, and externalises confidence as a badge — **green** (valid citations, strong grounding), **yellow** (weak grounding, verify), **red** (likely problem, manual verification required) — computed from the validator scores in `app/ui/components.py`.

25. Production Engineering Principles

Idempotency

Re-running ingestion/indexing must not duplicate data; documents are keyed by id and SHA-256, the vector store upserts, and re-runs converge.

Atomic writes

The registry is written to a temp file then renamed, so a crash cannot corrupt it.

Graceful degradation

Provider down $\rightarrow$ next provider; scan $\rightarrow$ OCR; validator error $\rightarrow$ mark unknown. Nothing crashes the request.

Observability

Per-stage latency and provider/fallback records are captured; an open-source LLM-observability platform (Langfuse) is wired for tracing.

Caching Translations and judge calls cache in SQLite to respect rate limits across runs.

Filtered ANN (future)

Pre-filter vs. post-filter trade-offs inform a planned per-regulation filtered search at larger scale.

26. Performance: Cold versus Warm Latency

Theory & measurement. The first question is slow because the model loads; the rest are fast. Measured on the running application: a *warm* query is $\sim 1.5\text{--}2.5$ s end-to-end, of which local retrieval is $\sim 110\text{--}160$ ms and the remainder is the LLM API round-trip. The *first* query after launch costs $\sim 7\text{--}9$ s because the ~ 2.3 GB FP32 embedder must load and run its first inference. Command-line invocations always pay this cold cost (each is a fresh process); the long-running application pays it once. Burst behaviour is bounded by queueing theory (Little’s law $L = \lambda W$; burst wait $\approx B/\Lambda - \tau$), which is why the P95 budget is 30 s rather than sub-second.

In Patih. Remedies, in order of preference: pre-warm the embedder at start-up (the app already background-initialises the generator; adding one dummy encode removes the first-query stall); complete the INT8 conversion to halve load time and memory; or move embedding to a cloud API (fast per query, but adds a network dependency and forces a re-index because the vector dimension changes from 1024).

27. Mathematical Reference

This section lists *every* mathematical symbol and equation the project uses. The equation numbers (E1)–(E31) are stable and are referenced from the Code-to-Mathematics map in §36, which states exactly which module implements which equation.

27.1 Symbols and notation

Symbol	Meaning (and where it is used)
$\phi(\cdot)$	The embedding function (text $\rightarrow$ vector); $\phi(q)$ for a query, $\phi(c)$ for a chunk. Dimension $\dim = 1024$.
q, c, d, t	Query; chunk/passage; document; term/token.
$\mathbf{h}_i, m_i$	The transformer’s hidden vector for token i , and its attention-mask bit (1 = real token).
$\mathbf{e}, \hat{\mathbf{e}}$	The mean-pooled embedding, and its L2-normalised (unit-length) form.
$\ \mathbf{v}\ $	Euclidean (L2) norm of vector $\mathbf{v}$; $\ \hat{\mathbf{e}}\ = 1$.
$\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^\top \mathbf{v}$	Dot (inner) product; equals cosine when both vectors are unit-length.
$f(t, D), D , \text{avgdl}$	Frequency of term t in document D ; length of D ; mean document length (BM25).
k_1, b	BM25 term-saturation and length-normalisation parameters.
$N, n(t), \text{IDF}(t)$	Corpus size; number of documents containing t ; inverse document frequency.
$r_i(c), k$	Rank of item c in ranked list i ; the RRF constant ($k = 60$).
$W, W_{\min}, W_{\max}$	A weight matrix and its min/max during INT8 calibration.
s, z	INT8 scale and zero-point.
z_i, T	The LLM’s logit for token i ; the decoding temperature ($T = 0.1$).
w, o	Child-window size (~ 1200 chars) and overlap (~ 150 chars) for reference-document chunking.
M, G	Set of model-produced citations; set of ground-truth (expected) citations.
P, R, F_1, J	Precision, recall, F_1 , Jaccard of M vs. G .
TP, FP	True/false positives (refusal precision).
$\mathcal{E}(\cdot), \mathcal{R}(\cdot)$	Set of legal <i>entities</i> / set of <i>relations</i> in a text; r = response, ctx = retrieved context.
p_0, c_ℓ, p_L	Base hallucination probability; catch-rate of validator layer ℓ ; residual after L layers.
AUC	Area under the ROC curve of the EG/RP discriminator.
$\lambda, \Lambda, \Lambda_p$	Request arrival rate; aggregate provider capacity; per-provider capacity (RPM).
U, Q, T_w	Active users/day; queries/user; working-window hours (arrival-rate model).
L, W, B, τ	Queue length; mean wait; burst size; refill window (queueing).
C, r, b_t	Token-bucket capacity; refill rate; bucket level at time t .
P_{50}, P_{95}, P_{99}	Latency percentiles.
$\mathcal{M}, \gamma, H, J(\pi)$	POMDP tuple; discount; horizon; expected return (Phase 3 agentic).
τ_{cache}	Cosine threshold for the Phase 2 semantic cache.

27.2 Equations

Eq.	Name and compact form	Role in Patih
(E1)	Mean pooling $\mathbf{e} = \frac{\sum_i m_i \mathbf{h}_i}{\sum_i m_i}$	Combine per-token outputs into one passage vector.
(E2)	L2 normalisation $\hat{\mathbf{e}} = \mathbf{e}/\ \mathbf{e}\ _2$	Unit-length vectors so cosine = dot product.
(E3)	Cosine similarity $\frac{\phi(q)^\top \phi(c)}{\ \phi(q)\ \ \phi(c)\ }$	Dense ranking over the e5-large space.
(E4)	Unit-vector identity $\cos(\mathbf{u}, \mathbf{v}) = \mathbf{u}^\top \mathbf{v}$ for $\ \mathbf{u}\ =\ \mathbf{v}\ =1$	Why Chroma’s dot-product search equals cosine.
(E5)	INT8 scale / zero-point $s = \frac{W_{\max}-W_{\min}}{255}, z = \text{round}(-W_{\min}/s)$	Calibrates FP32→INT8.
(E6)	INT8 transform $W^{\text{int8}} = \text{clip}(\text{round}(W/s) + z, -128, 127)$	Shrinks the embedder $\sim 4\times$ for CPU.
(E7)	BM25 $\sum_t \text{IDF}(t) \frac{f(t,d)(k_1+1)}{f(t,d)+k_1(1-b+b d /\text{avgdl})}$	Sparse ranker; recovers exact terms.
(E8)	IDF $\log \frac{N-n(t)+0.5}{n(t)+0.5}$	Down-weights frequent tokens.
(E9)	Reciprocal Rank Fusion $\sum_i \frac{1}{k+r_i(c)}, k=60$	Fuses dense + sparse ranks.
(E10)	Child windows $\# \approx \lceil (\ell - o)/(w - o) \rceil$	Number of overlapping windows for a section of length ℓ .
(E11)	Temperature softmax $e^{z_i/T} / \sum_j e^{z_j/T}, T=0.1$	Near-greedy, verbatim quotation.
(E12)	Precision / recall / F_1 $P = \frac{ M \cap G }{ M }, R = \frac{ M \cap G }{ G }, F_1 = \frac{2PR}{P+R}$	Per-question citation accuracy.
(E13)	Micro-recall $\sum_i M_i \cap G_i / \sum_i G_i $	Headline citation metric.
(E14)	Mean Jaccard $\frac{1}{N} \sum_i \frac{ M_i \cap G_i }{ M_i \cup G_i }$	Set-overlap diagnostic.
(E15)	Refusal precision $\text{TP}/(\text{TP}+\text{FP})$	Calibrated abstention.
(E16)	Entity Grounding $ \mathcal{E}(r) \cap \mathcal{E}(\text{ctx}) / \mathcal{E}(r) $	HalluGraph EG.
(E17)	Relation Preservation $ \mathcal{R}(r) \cap \mathcal{R}(\text{ctx}) / \mathcal{R}(r) $	HalluGraph RP.
(E18)	Discriminator AUC $\Pr[\text{score}(x^+) > \text{score}(x^-)]$	EG/RP separation (0.979 on legal docs).
(E19)	Defence in depth $p_L = p_0 \prod_{\ell=1}^L (1 - c_\ell)$	Multiplied catch rates bound residual hallucination.
(E20)	RAGAS Faithfulness $ \{c \in \mathcal{C}(a) : c \models \text{ctx}\} / \mathcal{C}(a) $	Claims entailed by context.
(E21)	RAGAS Answer Relevancy $\frac{1}{m} \sum_j \cos(\phi(q), \phi(q^{(j)}))$	Question–answer alignment.
(E22)	RAGAS Context Precision $\frac{1}{ R^+ } \sum_k \text{rel}_k \frac{\sum_{j \leq k} \text{rel}_j}{k}$	Ordering of relevant chunks.
(E23)	Latency percentile $P_q = \inf\{x : \hat{F}(x) \geq q/100\}$	P50/P95/P99 per stage.

Eq.	Name and compact form	Role in Patih
(E24)	Aggregated capacity $\Lambda = \sum_p \Lambda_p \approx 70\text{--}90$ RPM	Justifies four-tier fallback.
(E25)	Arrival rate $\lambda = UQ/(60 T_w) \approx 2.1$ RPM	100 users over 8 hours.
(E26)	Little’s law $L = \lambda W$	Queue-length vs. latency.
(E27)	Burst wait $W_{\text{burst}} \approx B/\Lambda - \tau$	When the P95 budget breaks.
(E28)	Token bucket $b_{t+\Delta t} = \min(C, b_t + \Delta t r) - 1$	Pre-emptive rate limiting.
(E29)	POMDP tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, T, R, \Omega, O, \gamma, H)$	Agentic-RAG formalisation (Phase 3).
(E30)	POMDP return $J(\pi) = \mathbb{E}_\pi \sum_{t=0}^H \gamma^t R(s_t, a_t)$	Objective for any agentic policy.
(E31)	Semantic-cache decision $\hat{a} = a^*$ if $\cos(\phi(q), \phi(q^*)) \geq \tau_{\text{cache}}$	Phase 2 cache reuse ($\tau_{\text{cache}} \approx 0.95$).

Part II

Technical Documentation and Build Guide

28. Project History: From a Single Document to Local-First

The system documented here is the settled state of a project that changed shape twice. Knowing the path explains several design choices that otherwise look arbitrary — in particular why a system advertised as “local” still calls a cloud LLM (§47), and why so much engineering went into surviving free tiers (§41).

28.1 Three phases

Single-document MVP

Phase 1 began as a minimum viable product over *one* regulation — Permen-sos 8/2023 (trafficking and problematic migrant workers) — to validate the whole pipeline against a small, well-understood document (~ 34 articles, ~ 13 pages) before scaling. The ingestion layer was designed multi-document-aware from the start (a `DocRegistry` keyed by `doc_id`, cross-reference fields), so only *one* document was indexed, not one document hard-wired. The application was originally called `chatbot_siti` and was renamed **Patih** on 2026-05-18.

Public-deployment attempt (abandoned)

The original target was a public URL serving ~ 100 active users per day (spread, not concurrent) from an always-free cloud VM behind a tunnel. Three attempts failed, in order: *(i)* Oracle Cloud Always-Free VM — the payment-method verification was declined at sign-up (even the free tier requires a card); *(ii)* a HuggingFace Space — flagged as abusive by anti-spam immediately after the push (a new account pushing large Git-LFS binaries quickly fits the abuse profile); *(iii)* a second, differently-named Space — flagged again, which confirmed the trigger was the *account-plus-binary-push* profile, not the Space name. After the second flag the public-deployment goal was dropped rather than further debugged.

Local-first pivot (current)

On 2026-05-28 the project pivoted to run on the laptop only, with the corpus growing through a folder watcher rather than a redeploy. The reasons were positive as well as defensive: *privacy* (the data never leaves the machine), *no flag risk*, and *full control* — and “100 concurrent users” was never the real requirement for what is, in honesty, a personal research assistant. The deploy step (Step 8) and the rate-limit endurance test (Step 9) were retired as no longer relevant.

28.2 Why the pivot mattered beyond hosting

The pivot was not merely “stop deploying”. Freed from the deployment blocker, the same sprint delivered the *multi-document* corpus that defines the system today: the local e5-large ONNX embedder, sidecar-driven (`<pdf>.meta.json`) ingestion, the multi-document indexer, the five parser fixes (§11), the generic ingester for article-less documents (§12), and the inbox watcher. The corpus grew from 1 document to 22 (§48) in the same pivot.

28.3 What survived, in case public hosting returns

The deployment work was not wasted and remains reusable: a FastAPI sidecar (`/health + /api/query`) mounted in the Chainlit ASGI app, a root Dockerfile following the HuggingFace convention (non-root user, port 8000), and a full green test suite were all completed and committed before the pivot. Should a public URL ever be wanted again, the recorded fastest path is a Cloudflare *tunnel* from the laptop (no card, no binary push to flag) rather than a cloud VM or a Space — and any exposed credential from the earlier attempts must be rotated first (§47).

29. System Overview and the Seven-Layer Architecture

Patih is seven layers of small single-purpose modules. Data flows *down* the layers when building the index (offline) and *across* them when answering a question (online).

Layer	Directory	Responsibility
1	<code>app/ingest/</code>	PDF → parsed, chunked, registered documents
2	<code>app/retrieval/</code>	Embed, index, and retrieve chunks (hybrid)
3	<code>app/llm/</code>	Language handling, prompting, multi-provider generation
4	<code>app/validators/</code>	Hallucination defence: citation, grounding, HITL
5	<code>app/ui/</code>	Chat interface (Chainlit) + REST sidecar (FastAPI)
6	<code>app/eval/</code>	Test set, metrics, evaluation reports
7	<code>tools/</code> , <code>infra</code>	Watcher, triage, smoke; observability (Langfuse)

29.1 The chain a request actually triggers

HTTP → sidecar → generator → retrieval → e5-large → LLM → validation → citation.

This is the exact path detailed in §37.

30. Why the Architecture Is Shaped This Way

Layered, single-purpose modules.

Each layer is tested and replaced independently — retrieval tuned without touching generation, a validator added without touching the UI. This makes the system buildable in stages and debuggable in isolation (and is why there are 307 targeted tests).

Offline indexing, online serving.

Expensive work (parsing, embedding) is done once and persisted; a query only reads prebuilt indexes, keeping per-query latency low and letting the corpus grow without touching serving.

Retrieval-heavy, LLM-light.

Following “retrieval is the ceiling” (§2.4), complexity lives in hybrid retrieval, parent expansion, and parsing; the LLM is a thin constrained summariser — the way to maximise verifiability under a free-tier model.

Defence in depth after generation.

LLMs hallucinate, so the answer is gated by independent cheap checks and weak answers are routed to a human.

A sidecar API beside the chat UI.

Mounting FastAPI inside the Chainlit ASGI app gives `/health` and `/api/query` for smoke tests, monitoring, and evaluation without a second server.

Local-first, cloud-LLM.

Data, index, embedding, and retrieval are local (privacy, no rate limits); only generation calls a cloud API, and documents grow via a folder watcher rather than a redeploy.

31. Technology and Tool Decisions

Every dependency was chosen against the constraints: zero monetary cost, CPU-only, Indonesian language, verifiability first, ~100 users/day (now single-user local).

Choice	Used for	Why this; what was rejected
RAG pattern	Overall approach	Verifiable, fresh, cheap; rejected raw LLM (hallucinates), long-context (Lost-in-the-Middle, cost), fine-tuning (cost, no citations).
multilingual-e5-large	Embeddings ($d=1024$)	Leads Indonesian retrieval benchmarks; English-only MiniLM scored worse. Considered and rejected BGE-M3 for Phase 1 simplicity.
ONNX + INT8 (target)	CPU inference	Portable fast CPU execution; INT8 to fit a laptop; FP32 in use until the Windows quantisation bug is retried.
Chroma	Vector store	Embedded, file-based, HNSW; heavier vector DBs (Qdrant/pgvector) deferred to Phase 2 when filtered ANN is needed.
rank_bm25 (BM25Okapi)	Sparse retrieval	Simple, dependency-light; no stemming (Sastrawi over-reduces legal terms).
RRF ($k=60$)	Fusion	Rank-only merge needs no calibration; a cross-encoder reranker (Cohere) is paid and was rejected.
LiteLLM router	LLM gateway	One OpenAI-compatible interface over many providers with fallback — essential for free-tier survival.

Choice	Used for	Why this; what was rejected
Groq / Gemini / Cerebras / Open-Router	LLM providers	Four free tiers chained; Llama 3.3 70B is the workhorse; small Indonesian-only models perform near-random on legal reasoning.
PyMuPDF	PDF text extraction	Fast, accurate native extraction with OCR fallback.
Tesseract (ind)	OCR	The realistic free, offline, Indonesian OCR.
lingua-py	Language detection	Reliable short-text ID/EN detection.
Chainlit	Chat UI	Python-native streaming chat with citation sidebar and feedback.
FastAPI (sidecar)	REST endpoints	Mounts inside Chainlit’s ASGI app; no second server.
SQLite	History & caches	Zero-config single-file storage.
RAGAS	LLM-as-judge eval	Reference-free Faithfulness/Answer-Relevancy; paid judge for a reliable one-off run.
Pydantic	Schemas	Typed validated data structures throughout.
Poetry / pytest	Env / tests	Reproducible pinned environment; 307 tests gate changes.

32. Complete Technology Stack

The previous section explained *why* the headline choices were made; this section is the *exhaustive inventory* of everything the project depends on, with versions and the role each plays. **Yes, Patih uses a vector database — Chroma** — listed under “Retrieval” below alongside every other component.

Technology	Version	Category — role in Patih
Python	3.11	<i>Runtime</i> — the implementation language.
Poetry	—	<i>Build</i> — reproducible, pinned dependency management.
llama-index-core	0.11.20	<i>RAG orchestration</i> — core RAG building blocks.
llama-index-embeddings-huggingface	0.3.1	<i>RAG</i> — HuggingFace embedding adapter.
llama-index-vector-stores-chroma	0.2.1	<i>RAG</i> — LlamaIndex↔Chroma adapter.
chromadb (Chroma)	0.5.15	Vector database — persistent HNSW index, cosine space; stores child-chunk embeddings (vector_store.py).
rank-bm25 (BM25Okapi)	0.2.2	<i>Sparse retrieval</i> — keyword index (bm25_store.py).
sentence-transformers	3.0.1	<i>Embedding</i> — model loading / dev fallback (MiniLM).
transformers	4.44.2	<i>Embedding</i> — tokeniser + model backbone.
torch (CPU build)	2.2.2	<i>Embedding</i> — tensor backend (CPU-only wheel).

Technology	Version	Category — role in Patih
onnxruntime	1.18.1	<i>Embedding</i> — runs the e5-large ONNX model on CPU.
optimum[onnxruntime]	≥1.22	<i>Embedding</i> — ONNX export + INT8 quantisation (<code>quantize_e5.py</code>).
huggingface-hub	0.24.6	<i>Embedding</i> — model download/cache.
intfloat/multilingual-e5-large	—	<i>Embedding model</i> — $d=1024$ multilingual encoder (the actual weights).
litellm	1.45.0	<i>LLM gateway</i> — OpenAI-compatible router + fallback (<code>gateway.py</code>).
google-generativeai	0.7.2	<i>LLM</i> — native Gemini SDK (embedding + fallback path).
tenacity	8.5.0	<i>LLM</i> — retry/backoff helper.
Groq / Gemini / Cerebras / OpenRouter	—	<i>LLM providers (services)</i> — four free tiers; Llama 3.3 70B (Groq) is the workhorse, Qwen 3 (Cerebras), Gemini 2.5 Flash.
pymupdf	1.24.10	<i>PDF</i> — primary native-text extraction (<code>pdf_loader.py</code>).
unstructured[pdf]	0.15.13	<i>PDF</i> — structured-extraction option.
pdfplumber	0.11.4	<i>PDF</i> — backup parser.
pytesseract	0.3.13	<i>OCR</i> — Python binding to Tesseract.
Tesseract OCR (binary) + ind pack	5.x	<i>OCR (system)</i> — reads scanned PDFs in Indonesian (<code>models/tessdata/</code>).
chainlit	1.2.0	<i>UI</i> — conversational web app (<code>chainlit_app.py</code>).
FastAPI + Uvicorn	(via chainlit)	<i>UI / API</i> — the mounted REST sidecar (<code>api_sidecar.py</code>) and ASGI server.
ragas	0.2.6	<i>Evaluation</i> — LLM-as-judge Faithfulness / Answer-Relevancy.
datasets	2.21.0	<i>Evaluation</i> — dataset plumbing for RAGAS.
langfuse	2.51.5	<i>Observability</i> — LLM tracing.
lingua-language-detector	2.0.2	<i>NLP</i> — ID/EN/mixed language detection (<code>lang_detect.py</code>).
pydantic	2.9.2	<i>Schemas</i> — typed records (ChunkMeta, registry, results).
pydantic-settings	2.5.2	<i>Config</i> — settings loading.
structlog	24.4.0	<i>Logging</i> — structured logs.
python-dotenv	1.0.1	<i>Config</i> — loads <code>.env</code> (keys, <code>EMBEDDER_BACKEND</code>).
tqdm	4.66.5	<i>Utility</i> — progress bars.
orjson	3.10.7	<i>Utility</i> — fast JSON.
SQLite	(stdlib)	<i>Storage</i> — conversation history, translation cache, judge cache.
JSON	(stdlib)	<i>Storage</i> — parsed documents, registry, parent-lookup, metadata sidecars.
Pickle	(stdlib)	<i>Storage</i> — the BM25 index.

Technology	Version	Category — role in Patih
ONNX	—	<i>Storage/format</i> — the serialised embedder.
pytest (+asyncio,+cov)	8.3.3	<i>Dev/test</i> — 307-test suite.
hypothesis	6.112.1	<i>Dev/test</i> — property-based tests for the parser.
ruff / black / mypy	—	<i>Dev</i> — lint / format / type-check.
Git + Git-LFS	—	<i>Dev/infra</i> — version control; LFS for model/index binaries.

Storage at a glance. Embeddings live in **Chroma** (`data/chroma/`); the keyword index is a **pickle** (`data/bm25/`); parsed documents, the registry, and the parent-lookup are **JSON**; conversation history and caches are **SQLite**; the embedder is an **ONNX** file (`models/`). There is no external database server — every store is a local file, consistent with the local-first design.

33. Data Schemas

The system passes a few well-defined records between layers. Understanding them is most of understanding the code.

ChunkMeta — (`app/ingest/chunker.py`) The unit of retrieval. Fields: `chunk_id`, `parent_id`, `doc_id`, `chunk_type` (child/parent); structure — `bab`, `bagian`, `pasal` (nullable), `ayat`, `huruf`, `section_title` (for reference docs); text — `text` (shown to the LLM) and `text_for_embed` (with document label and "passage: " prefix); provenance — `source_page`, `source_pdf_path`, `version`; and tags (e.g. `always_on`, `reference`). Making `pasal` nullable is what lets one schema serve both regulations and reference documents.

DocRegistry — (`app/ingest/doc_registry.py`) One record per document: `doc_id`, `title`, `nomor`, `tahun`, `jenis_regulasi` (an enum: `PERMENSOS/UU/PP/PERPRES/PERBUP/PERMENKES/PERMEN_KEMENAKER/PERBANK/PERMA/OTHER`), `judul_lengkap`, `tentang`, `dates` and `status`, `pdf_path`, `pdf_sha256` (idempotency), `parse_method`, `parse_version`, `quality_flags`, and counts (`n_pasal`, `n_chunks_child`, `n_chunks_parent`). Persisted to `data/registry/documents.json`.

RetrievalResult — (`app/retrieval/pipeline.py`) `parent_chunks` (the ordered context dicts) plus a `debug` block (`dense/sparse/fused` top-lists, expanded parents, cross-ref additions, final article order, elapsed milliseconds).

GenerationResult — (`app/llm/generator.py`) `response`, `response_lang`, `retrieved_pasals`, `parent_chunks` (carried for validation and citation cards), `llm_provider_used`, `model_name_used`, `fallback_chain_attempts`, `query_lang`, `query_translated`, `latency_ms` (retrieval/llm/total), `tokens_in/out`, and `validation` (a serialised `ValidationResult` or null).

ValidationResult — (`app/validators/pipeline.py`) `citations_extracted`, `citations_valid`, `citation_accuracy`, `eg_score`, `rp_score`, `hitl_flag`, `hitl_reasons`, `is_refusal`, and a `debug` block.

EvalQuestion — (`app/eval/testset_loader.py`) `qid`, `question`, `tier`, `question_type`, `language`, `expected_pasal_refs`, `must_refuse`.

34. Configuration and Parameter Reference

Every tunable in one place (defaults shown).

Parameter	Value	Where / why
Embedder model	e5-large	multilingual, $d=1024$, ONNX FP32 on CPU
EMBEDDER_BACKEND	onnx	pin so index and query share a dimension
Max token length	512	embedder tokeniser truncation
Index batch size	16	children per embed batch
top_k_dense	15	dense candidates
top_k_sparse	15	BM25 candidates
top_k_fused	8 (12 in eval)	RRF output expanded to parents
RRF k	60	fusion constant
Cross-ref cap	3	max referenced articles added
Always-on article	Pasal 1	dominant document's definitions
Child window (reference)	~1200 chars	generic ingester
Child overlap (reference)	~150 chars	generic ingester
Penjelasan cut threshold	40%	only strip if heading past 40% of doc
LLM temperature	0.1	near-greedy, verbatim quotation
LLM max tokens	1500	answer length cap
Provider RPM (G/Gr/C/OR)	10/30/30/20	token-bucket capacities
Retry per attempt / budget	~5 s / ~15 s	tail-latency control
EG threshold	0.95	below $\Rightarrow$ HITL flag
RP threshold	0.85	below $\Rightarrow$ HITL flag
Citation accuracy target	$\geq 90\%$	acceptance threshold
P95 latency budget	≤ 30 s	acceptance threshold
OCR render scale	2 $\times$	page-image resolution for Tesseract
OCR language	ind	Indonesian Tesseract model

35. Component Catalogue — Every Module and How It Works

No source code is shown; each entry is “what the file is responsible for and how it does it”.

35.1 Layer 1 — Ingestion (app/ingest/)

pdf_loader.py — Extracts text. Tries PyMuPDF native extraction; assesses quality (chars/page, non-ASCII ratio); if poor or forced, renders each page at 2 $\times$ and runs Tesseract (**ind**). A helper auto-locates the binary and language data so OCR is unattended. Returns full text plus metadata (pages, parse method, quality).

structure_parser.py — Builds the BAB/Bagian/Pasal/ayat/huruf AST with regexes; applies the five fixes of §11 (Penjelasan strip, omnibus signal, pre-BAB recovery, period tolerance, cross-chapter de-dup). Zero parsed articles signals “not a regulation”.

chunker.py — Defines **ChunkMeta** and emits parents (Pasal) and children (ayat/huruf) with stable ids, parent links, page numbers, embed text, and the **always_on** tag on Pasal 1.

doc_registry.py — Defines **DocRegistry**; computes SHA-256; writes the registry atomically (temp-then-rename); updates a document in place on re-ingest.

validators.py — Parse-time quality checks (no null article, parents resolve, hash matches, minimum article count, no duplicate ids); returns flags rather than raising.

cli.py — Ingest entry point for one regulation; loads metadata from a `<pdf>.meta.json` sidecar; runs `load→parse→chunk→validate→persist`.

generic_ingest.py — The non-article ingester: heading/page sections, size-windowed children, same schema with no article number and a section title.

35.2 Layer 2 — Retrieval (`app/retrieval/`)

embedder.py — Encodes text; three backends pinned by `EMBEDDER_BACKEND`; for ONNX it tokenises, runs the model, mean-pools, and L2-normalises; adds the e5 query prefix.

vector_store.py — Wraps Chroma (cosine space): upsert (vector + text + scalar metadata, list fields JSON-encoded), dense query (optionally document-filtered), and fetch by id/metadata.

bm25_store.py — Builds, pickles, and queries the BM25Okapi keyword index; no-stemming tokeniser.

hybrid.py — RRF fusion of the dense and sparse lists into the top-N.

parent_expander.py — Maps fused children to parents via `parent_id`, de-duplicates, keeps best score.

cross_ref_resolver.py — Finds “Pasal X” references in retrieved parents and pulls those articles in (capped at 3), resolving against the *same* document.

always_on.py — Ensures Pasal 1 of the dominant retrieved document is present.

pipeline.py — The orchestrator: lazy-loads embedder/BM25/Chroma/parent-lookup and a merged child-lookup across all parsed docs; per query embeds, dense+sparse searches (optional `doc_filter`), fuses, expands, resolves cross-refs, prepends definitions, returns ordered context + debug.

indexer.py — Builds indexes from parsed JSON: globs all parsed docs, de-duplicates chunk ids, embeds children, upserts to Chroma, rebuilds BM25 from the union, merges parent-lookup; supports `--rebuild` and an incremental single-document mode.

35.3 Layer 3 — LLM Gateway and Generation (`app/llm/`)

lang_detect.py — ID/EN/mixed detection (lingua-py) with confidence threshold and ID default.

translator.py — EN→ID query translation for retrieval; SQLite-cached; graceful fallback.

prompts.py — Loads the system prompt; per-document context headers (article or section/page); builds the chat messages.

rate_limiter.py — Per-provider token buckets and a registry; blocks to stay under each RPM cap.

gateway.py — LiteLLM router with the four providers in priority order and a fallback chain; records per-attempt latency and provider used.

generator.py — End-to-end orchestrator and the object the sidecar calls: detect language, translate, retrieve, build prompt, call gateway, extract context articles, run validation, queue HITL, return `GenerationResult`.

cli.py — Ask one question from the command line (cold each run; see §26).

configs/prompts/*.md — The Indonesian system prompt (answer-and-cite rules including the reference-document rule), a bilingual variant, and few-shot examples.

35.4 Layer 4 — Validators (`app/validators/`)

citation_extractor.py — Regex extraction of every “Pasal N [ayat (M)] [huruf x]” with offsets.

whitelist_validator.py — Range check + Chroma existence; valid if found in any retrieved document; huruf fallback confirms the letter appears in the ayat text.

entity_grounding.py — HalluGraph EG: fraction of legal terms in the answer also in context.

relation_preservation.py — HalluGraph RP: per-sentence content overlap with context, skipping footers/citation-only lines.

pipeline.py — Orchestrates the four; refusal detection; threshold gate (invalid citation / EG<0.95 / RP<0.85) sets the HITL flag and reasons; appends flagged cases to `data/hitl_queue.jsonl`; returns `ValidationResult`.

cli.py — Validate an arbitrary (query, response) pair.

35.5 Layer 5 — User Interface (`app/ui/`)

chainlit_app.py — The chat app: welcome + background generator pre-warm; per message a thinking placeholder → generator → badge + answer → citation cards → persist → feedback buttons; also mounts the FastAPI sidecar.

components.py — Confidence badge (green/yellow/red from validator scores) and citation cards (per document; “Pasal X — *label*” or “*section* — *label*”, with the full text and a page/document source line).

history.py — SQLite conversation store (turns + thumbs feedback).

api_sidecar.py — FastAPI mounted in Chainlit: `/health` and `/api/query` (question → generator → answer with citations, validity flags, scores, latency); import-light so it is unit-testable without Chainlit.

35.6 Layer 6 — Evaluation (`app/eval/`)

testset_loader.py — Loads the tiered, labelled question set into a typed schema.

citation_accuracy.py — Precision/recall/F1/Jaccard at article/paragraph/letter granularity, micro and per-tier.

refusal_precision.py — Refusal detection (ID/EN templates) and precision over must-refuse questions.

latency_meter.py — P50/P95/P99 per stage.

ragas_runner.py — RAGAS Faithfulness/Answer-Relevancy judges with SQLite cache, throttling, and NaN filtering.

ratelimit_endurance.py — Steady-rate throughput harness.

report.py / cli.py — Markdown report (metrics vs. thresholds, per-tier, worst questions, latency, recommendation) and `run/subset/endurance` subcommands.

35.7 Layer 7 — Tooling and Observability

`tools/triage_pdfs.py` — Classifies a folder of PDFs into regulation vs. reference and infers type/number/year.

`tools/generate_meta_sidecars.py` — Auto-drafts a `.meta.json` per regulation; de-duplicates by SHA-256.

`tools/bulk_ingest.py` — Ingests every document in a manifest, skipping hash-matched ones; reports per-document results.

`tools/inbox_watcher.py` — The folder-watcher daemon: polls `data/raw/inbox/`; on a stable `<pdf>+.meta.json` pair, moves them to `data/raw/`, routes by `doc_type` (reference → generic ingester, else regulation parser), ingests, indexes incrementally, quarantines failures; a lock file prevents concurrent runs.

`tools/smoke_multidoc.py` — Counts chunks per document and checks sample questions route to the expected source.

`deploy/scripts/quantize_e5.py` — Exports and INT8-quantises the embedder (FP32 fallback).

36. Code-to-Mathematics Map — Which Equation Each Module Implements

This table answers, for every module, “which theory and which equation does this code apply, and how”. Equation numbers refer to §27.2; section references point to the theory in Part I. Modules that are pure orchestration or I/O (no mathematics of their own) are marked “— (control/IO)”.

Module	Equations	Theory	How the maths is applied
<code>embedder.py</code>	E1, E2, E3, E4	§4	Mean-pool the model’s token vectors (E1), L2-normalise (E2); the unit-vector identity (E4) lets similarity be a dot product (E3).
<code>quantize_e5.py</code>	E5, E6	§5	Calibrate and apply the FP32→INT8 weight transform (FP32 fallback when it fails).
<code>vector_store.py</code>	E3, E4	§6	Chroma cosine-space query; dot product over unit vectors via HNSW ANN.
<code>bm25_store.py</code>	E7, E8	§7	BM25Okapi scoring with IDF weighting over no-stemming tokens.
<code>hybrid.py</code>	E9	§8	Reciprocal Rank Fusion of the dense and sparse lists, $k = 60$.
<code>parent_expander.py</code>	set ops (E12-style $\cap$)	§9	De-duplicate children to parents; keep the maximum child score per parent.
<code>cross_ref_resolver.py</code>	— (set membership)	§20	Per-document reference resolution; cap of 3 additions.
<code>always_on.py</code>	— (set membership)	§10, §20	Ensure Pasal 1 of the dominant document is in the set.
<code>chunker.py</code>	— (structural)	§10	Builds parent/child units; no scoring maths.

Module	Equations	Theory	How the maths is applied
<code>generic_ingest.py</code>	E10	§12	Window count for a section of length ℓ given size w , overlap o .
<code>structure_parser.py</code>	— (regex/AST)	§11	Pattern matching; the 40% Penjelasan rule is a position ratio, not an equation.
<code>pdf_loader.py</code>	— (thresholds)	§13	Quality thresholds (chars/page, non-ASCII ratio) decide native-vs-OCR.
<code>prompts.py</code>	E11	§15	The generation temperature $T = 0.1$ enters the softmax (E11).
<code>gateway.py</code>	E11, E24, E26, E27	§16, §26	Decoding temperature; aggregate capacity and queueing bounds shape the fall-back order and retry budget.
<code>rate_limiter.py</code>	E28	§16	Per-provider token bucket (capacity C , refill r).
<code>generator.py</code>	— (orchestration)	§15	Sequences retrieval→prompt→LLM→validation; no maths of its own.
<code>lang_detect.py</code>	— (classifier)	§17	Confidence-thresholded language ID.
<code>translator.py</code>	— (IO/cache)	§17	EN→ID query translation; cached.
<code>citation_extractor.py</code>	— (regex)	§18	Pattern extraction of citations (feeds E12–E15).
<code>whitelist_validator.py</code>	E12 (sets)	§19	Set membership of citations in the retrieved documents; range check.
<code>entity_grounding.py</code>	E16	§19	Entity-Grounding ratio.
<code>relation_preservation.py</code>	E17	§19	Relation-Preservation ratio.
<code>validators/pipeline.py</code>	E15, E18	§19	Defence-in-depth threshold gate over EG/RP and citation validity (E18 motivates the gate).
<code>components.py</code>	E16, E17	§24	Badge colour from the EG/RP scores and thresholds.
<code>citation_accuracy.py</code>	E12, E13, E14	§23	Precision/recall/ F_1 , micro-recall, mean Jaccard.
<code>refusal_precision.py</code>	E15	§23	Refusal precision on must-refuse questions.
<code>ragas_runner.py</code>	E20, E21, E22	§23	RAGAS Faithfulness / Answer-Relevancy / Context-Precision judges.
<code>latency_meter.py</code>	E23	§26	P50/P95/P99 percentiles per stage.
<code>ratelimit_endurance.py</code>	E24, E25, E26	§26	Steady-rate arrival (λ) vs. capacity (Λ), Little’s law.
<i>(Phase 3 agentic)</i>	E29, E30	§21	POMDP formalisation of a bounded reasoning loop.
<i>(Phase 2 cache)</i>	E31	§45	Semantic-cache reuse threshold.

Reading the map: a researcher can start from any equation in §27.2 and find the module that realises it; a developer can start from any module in §35 and find the theory (Part I) and the precise formula it computes.

37. The Online Query Flow, Step by Step

Expanding *HTTP* → *sidecar* → *generator* → *retrieval* → *e5-large* → *LLM* → *validation* → *citation*:

1. **HTTP.** The browser (Chainlit) or an API client sends the question; API clients POST to `/api/query`.
2. **Sidecar.** `api_sidecar.py` validates the request and calls the generator.
3. **Generator.** `generator.py` detects language; for English it translates the query for retrieval.
4. **Retrieval + e5-large.** `pipeline.py` embeds the query with e5-large (`embedder.py`), runs dense (Chroma) and sparse (BM25) search, fuses with RRF, expands children to parent articles, resolves per-document cross-references, and prepends the dominant document’s definitions article.
5. **LLM.** `prompts.py` builds the system+user messages with per-document-labelled context; `gateway.py` calls the first available provider, falling back as needed.
6. **Validation.** `validators/pipeline.py` extracts citations, whitelists them against the retrieved documents, and scores EG and RP; the threshold gate sets the HITL flag and badge colour.
7. **Citation.** `components.py` renders the badge and click-through citation cards; the turn is stored and feedback buttons shown.

Warm latency is dominated by the LLM round-trip; local retrieval is ~ 150 ms (§26).

38. The Offline Ingestion Flow, Step by Step

1. **Drop** `<doc>.pdf` and `<doc>.pdf.meta.json` into `data/raw/inbox/`.
2. **Detect & route.** `inbox_watcher.py` sees the stable pair, moves it to `data/raw/`, routes by `doc_type`.
3. **Load.** `pdf_loader.py` extracts text (OCR for scans).
4. **Parse / chunk.** Regulations → `structure_parser.py` (with the five fixes) then `chunker.py`; reference documents → `generic_ingest.py`.
5. **Register.** `doc_registry.py` records the document and hash atomically.
6. **Index (incremental).** `indexer.py` embeds new children, upserts to Chroma, rebuilds BM25 and parent-lookup from the union.

39. Validation and Evaluation in Practice

39.1 Per-answer validation (always on)

Every answer passes Layer 4 before display: citations extracted and checked for existence in the retrieved documents; EG and RP scored; the threshold gate (invalid citation, or $EG < 0.95$, or $RP < 0.85$) raises the HITL flag, colours the badge, and queues the case. This is the live safety net.

39.2 Batch evaluation (offline, labelled set)

A 50-question set, tiered by difficulty (definitional, extraction, cross-reference, out-of-scope, adversarial), with expected articles, drives the metrics of §23. Acceptance thresholds: citation accuracy $\geq 90\%$; Faithfulness and Answer-Relevancy ≥ 0.85 ; RP mean ≥ 0.80 ; EG mean ≥ 0.90 ; refusal precision $\geq 80\%$; P95 latency ≤ 30 s; hard-failure rate $< 5\%$.

40. Evaluation Trajectory (v1–v6)

Real runs as Phase 1 progressed; each shift traced to a code change.

Metric	Threshold	v4	v5	v6
RP mean (answered)	≥ 0.80	0.91	0.98	0.98
EG mean (answered)	≥ 0.90	1.00	0.998	1.00
Refusal precision	$\geq 80\%$	100%	100%	100%
P95 latency	≤ 30 s	7.7 s	4.8 s	4.3 s
Hard-failure rate	$\leq 5\%$	13/50	0/50	0/50
Citation accuracy	$\geq 90\%$	81.8%	75.4%	78.7%
RAGAS Faithfulness	≥ 0.85	nan	nan	nan
RAGAS Answer Relevancy	≥ 0.85	nan	nan	nan

Lessons. v4: skipping the LLM’s bulleted footer citations lifted RP 0.67→0.91. v5: first clean run, 0/50 hard failures with Llama 3.3 70B on all questions; citation accuracy diagnosed as a model limit, not retrieval. v6: three new prompt rules and `top_k_fused=12` gave +3.3 pp micro-recall, then tuning saturated — further headroom needs a stronger model, per-*angka* chunking of Pasal 1, or test-set curation. The RAGAS judges never completed on free judge tiers (returning NaN); a one-off paid judge run is deferred to pre-launch. **This batch evaluation has not yet been re-run on the multi-document corpus** — the main open evaluation item, since some once-out-of-scope questions are now answerable from another law.

41. Free-Tier Economics and the Free-Judge Problem

The zero-budget constraint (§31) put every LLM call onto a free tier, and the *real* free-tier limits differ sharply from the published or assumed ones. Discovering them the hard way — during evaluation — reshaped the architecture (it is why Groq, not Gemini, became the workhorse) and produced the one acceptance metric that still has no number. This section records the economics so a rebuilder does not relearn them by surprise.

41.1 The real limits versus the assumed ones

The binding limit is rarely requests-per-minute; it is the *daily* budget — requests per day (RPD) or tokens per day (TPD) — which a burst of evaluation calls exhausts in minutes even though the steady serving rate (§16, $\lambda \approx 2.1$ RPM) would be comfortable.

Provider model	/ Assumed free tier	Real limit	binding	Consequence
Gemini 2.5 Flash	10 RPM / 1500 req/day	~5 RPM and ~20 req/day (RPD)		The 1500/day figure was stale (an older API tier). 20/day cannot serve or evaluate; Gemini was <i>demoted</i> from primary to a fallback/judge role.
Groq Llama 3.3 70B	30 RPM	30 RPM and ~100k TPD	and	At ~3.4k tokens/query (~3k context-in + ~400 out), a 50-question run ≈170k tokens > 100k ⇒ mid-run exhaustion. Still the best free option, so it became the workhorse.
Cerebras Qwen 3 32B	30 RPM / 1M tokens/day	As stated, but the small llama3.1-8b is too weak		Usable as a fallback <i>generator</i> ; unusable as a structured <i>judge</i> (below).
OpenRouter (:free)	20 RPM, several free models	deepseek-r1:free returns 404; hermes-3-405b:free daily quota ≈0		The fourth fallback tier is effectively empty; retained in config for when a working free slug reappears (§16).

The arithmetic that matters: aggregate *serving* capacity across the four tiers is $\Lambda \approx 70\text{--}90$ RPM (E24), far above the expected arrival rate $\lambda \approx 2.1$ RPM for 100 users over an 8-hour day (E25) — so *serving* is fine. *Evaluation*, by contrast, is a burst of 100 calls (50 generations + 50 judge calls) issued back-to-back, which is exactly the shape a daily RPD/TPD cap cannot absorb. The mismatch between a gentle steady rate and a spiky evaluation load is the root cause of the recurring “quota cascade” in §40.

41.2 The free-judge problem

RAGAS Faithfulness and Answer-Relevancy (E20–E21) are *LLM-as-judge* metrics: each of the 50 questions needs a judge call per metric, i.e. up to 100 judge calls for one evaluation. No free judge backend ever completed the set:

Gemini judge

20 RPD ⇒ roughly 1 of 100 calls succeeds before the daily quota is gone.

Cerebras Llama-3.1-8B

fast and within quota, but **too weak for structured judging**: 50–76% of its outputs fail to parse into the RAGAS schema, so most records return nothing usable.

OpenRouter DeepSeek-R1 :free

discontinued (404).

OpenRouter Hermes-3-405B :free

daily quota effectively zero (0 of 392 attempts succeeded).

The combined result is that Faithfulness and Answer-Relevancy stayed **nan** across every run v1–v6 (§40) — not because the answers are unfaithful, but because no affordable judge could score them. (A latent bug compounded the confusion: pandas **NaN** from failed judge calls leaked into the aggregate and hid the few partial-success records; `ragas_runner.py` now filters **NaN** before averaging.)

41.3 The decision, and the migration trigger

The engineering time spent coaxing a free judge exceeded the cost of simply paying for one. The recorded decision is therefore to **stop debugging free judges** and run RAGAS *once* with a paid Gemini judge — on the order of \$0.05 for the whole 50-question set — immediately before any wider launch. More generally, the free tier is adequate for a single-user local assistant but *not* for a real ~ 100 -users/day production target or for reliable evaluation: the migration trigger is a sustained daily active user count, or a need for trustworthy judge numbers, at which point a modest $\sim \$5$ /month tier (Groq Dev, or paid Gemini Flash) removes both the TPD cliff and the free-judge problem at once.

42. How We Know the App Is Ready

Readiness was judged by evidence:

1. **Automated tests.** 307 tests pass (one pre-existing placeholder aside) across parsing, chunking, fusion, validators, citation extraction, UI components, and the watcher.
2. **Retrieval routing verified.** The multi-document smoke test confirms questions reach the right law (child-protection $\rightarrow$ the child-protection law; human-rights $\rightarrow$ the human-rights law; trafficking $\rightarrow$ the anchor regulation; development priorities $\rightarrow$ the RPJMN summary).
3. **End-to-end through the running app.** `/health` returned healthy; `/api/query` returned a correct, well-cited answer with valid-citation flags and no HITL flag; a reference-document question was answered with a document-and-page citation, not a fabricated article.
4. **Performance measured.** Warm queries ~ 1.5 – 2.5 s (retrieval ~ 150 ms); the only slow path is the one-time cold start.

Honest caveats. Not yet done: a manual click-through of the browser chat widget and citation sidebar, and the multi-document batch re-evaluation plus the RAGAS judge run. Accuracy is a *calibrated* target ($\geq 90\%$ with citations and human review), *not* a guarantee — for any consequential legal use, verify the cited article against the original text. *Verdict:* ready as a personal local research assistant; for wider or official use, finish the browser smoke test and the multi-document re-evaluation.

43. Build From Scratch — Chronological Steps

Each step yields something testable before the next. Effort is a rough solo estimate.

1. **Scaffold** (~ 2 h). Create the layered directories; pin dependencies with Poetry (LlamaIndex/Chroma/LiteLLM/Chainlit/rank_bm25/PyMuPDF/RAGAS/Pydantic); verify core imports.
2. **Ingestion** (~ 8 – 12 h). Build `pdf_loader` (OCR fallback), `structure_parser` (AST + the five fixes), `chunker`, `doc_registry`, parse-time validators, the ingest cli, and `generic_ingest`. *Test:* parse the anchor regulation; article count matches its table of contents.
3. **Retrieval + embedding** (~ 10 – 15 h). Build `embedder`, `vector_store`, `bm25_store`, `hybrid`, `parent_expander`, `cross_ref_resolver`, `always_on`, `pipeline`, `indexer`. *Test:* a golden recall smoke test retrieves the expected articles.
4. **LLM gateway + generator** (~ 8 – 12 h). Build `lang_detect`, `translator`, `prompts`, `rate_limiter`, `gateway`, `generator`. *Test:* a query returns a cited answer within the latency budget.

5. **Validators** (~6–8 h). Build `citation_extractor`, `whitelist_validator`, `entity_grounding`, `relation_preservation`, the validation pipeline with the HITL queue. *Test*: a good answer passes; an adversarial one is flagged.
6. **UI** (~6–8 h). Build `chainlit_app`, `components`, `history`, `api_sidecar`. *Test*: `/health` and `/api/query` respond.
7. **Evaluation** (~15–20 h). Build the test-set loader and metric modules; run the batch evaluation; read the report.
8. **Multi-document scale-up**. Triage a folder of PDFs; generate sidecars; bulk-ingest regulations; ingest reference documents with the generic ingester; rebuild the index. *Test*: the multi-document smoke test routes correctly.
9. **Growth path**. Run the folder watcher; drop new PDFs into the inbox to ingest them automatically.

44. Trade-offs Explicitly Accepted

Trade-off	Choice	Rejected	Reason
Citation discipline	Free model + validators	Claude Sonnet	paid
Multi-doc reasoning	Hybrid retrieval + cross-ref	GraphRAG now	premature; cost not justified yet
Reranking	RRF + parent expansion	Cohere Rerank API	paid; sufficient without
Embedder hosting	e5-large ONNX (FP32)	paid embedding API	free tier rate-limited; CPU self-host viable
Agentic loop	none in Phase 1	Self-RAG / CRAG now	compounding-hallucination cautions
HITL	JSONL flag queue	full reviewer dashboard	personal MVP; later phase
Auth	none (single-tenant)	OAuth multi-tenant	local/personal use
Deployment	local laptop	public cloud URL	deploy attempts failed; audience is local
Latency budget	$P95 \leq 30\text{ s}$	sub-second	free-tier limits force queueing
INT8	FP32 (for now)	blocked by Windows bug	retry with UTF-8 console

45. Roadmap and Open Decisions

45.1 Phase 2 — breadth and structure

Ingest more regulations (incl. UU 21/2007 TPPO, UU 18/2017 PMI, PP 59/2021); adopt Summary-Augmented Chunking against DRM; build a Pasal–ayat–reference knowledge graph and a GraphRAG retrieval path for thematic queries; migrate to a filtered-ANN vector DB (Qdrant/pgvector) when per-regulation filtering is needed; add a query-complexity classifier; extend the eval set to 100 cross-document questions; move to a paid LLM tier once free RPM is exhausted.

45.2 Phase 3 — agentic and production HITL

Bounded multi-step reasoning (Self-RAG/CRAG/Chain-of-Retrieval, ≤ 3 hops, guarded by the agentic-RAG cautions); replace the regex EG/RP with an NLI-based verifier; a full HITL reviewer dashboard; a compliance audit trail (Personal Data Protection / UU 27/2022); and a possible move to a stronger model tier.

45.3 Open decisions

1. Knowledge-graph technology for Phase 2 (Neo4j vs. Postgres-as-KG).
2. Whether to re-attempt INT8 (and pre-warm) now, given the cold-start cost.
3. When to re-run the full evaluation on the multi-document corpus (the main open quality question).
4. Whether to ingest the deferred statistics books / scanned SOP, given their low retrieval value.

46. Repository Layout and Operations

46.1 Where things live

Path	Contents
app/ingest/	Layer 1 ingestion
app/retrieval/	Layer 2 retrieval + indexing
app/llm/	Layer 3 language, prompting, generation
app/validators/	Layer 4 hallucination defence
app/ui/	Layer 5 Chainlit app + FastAPI sidecar
app/eval/	Layer 6 evaluation
configs/prompts/	System / bilingual / few-shot prompts
data/raw/	Source PDFs (+ inbox/ watched folder)
data/parsed/	Parsed JSON per document
data/chroma/	Vector store
data/bm25/	BM25 index + parent-lookup
data/registry/	Document registry
models/	ONNX embedder + Tesseract language data
tools/	Triage, sidecars, bulk ingest, watcher, smoke
tests/	Unit, integration, golden

46.2 Run it

```
# Terminal 1 -- start the assistant, then open http://localhost:8000
.\.env\Scripts\chainlit.exe run app/ui/chainlit_app.py --port 8000

# Terminal 2 (optional) -- auto-ingest documents dropped into the inbox
.\.env\Scripts\python.exe -m tools.inbox_watcher

# Rebuild the whole index from parsed documents
.\.env\Scripts\python.exe -m app.retrieval.indexer --rebuild
```

46.3 Add a document

Copy the PDF to `data/raw/inbox/` and create `<name>.pdf.meta.json`; for a regulation set `jenis_regulasi`, and for a free-form document add `"doc_type": "reference"`. The watcher ingests and indexes it within $\sim 10\text{--}60$ s; failures land in `data/raw/failed/` with a reason.

47. Privacy, Security, and the Operational Runbook

“Local-first” (§28) is a precise claim, not a blanket one. This section states exactly what stays on the machine and what leaves it, how credentials are handled, and what happens — and how to recover — when each part fails.

47.1 The privacy model: what is local, what leaves the machine

Everything except generation is local. The source PDFs, the parsed JSON, the Chroma vector store, the BM25 index, the e5-large embedder, the conversation history, and the translation/judge caches all live on the laptop and never leave it. *One* step reaches the network: the LLM call. The query — translated to Indonesian first if it arrived in English — together with the *retrieved article text* is sent over HTTPS to a cloud provider (Groq / Gemini / Cerebras / OpenRouter). So “local-first” means **data, embeddings, retrieval, and indexing are local; inference is cloud**.

Why this is acceptable here, and when it would not be. The corpus is *published public regulation*, so transmitting article text to a third-party model leaks nothing secret. What *does* leave the machine is the user’s *question* and the particular *combination* of articles retrieved for it — which is low-risk for general regulatory questions but would matter the moment a query contained personal data or a confidential case. A fully-local alternative (a local LLM via Ollama) was considered and rejected on *quality*, not effort: small Indonesian-capable models score near-random on legal reasoning (Koto et al. 2024, §50), and the whole design rests on the LLM being a disciplined citer. The privacy/quality trade-off is thus explicit, and a personal-data audit trail under the Indonesian PDP law (UU 27/2022) is a Phase 3 item (§45) for any multi-tenant or sensitive use.

47.2 Secret handling

All credentials live in a `.env` file that is `.gitignored`; the committed `.env.example` is the template, and `python-dotenv` loads the real values at start-up. The secrets are the four provider keys (`GEMINI_API_KEY`, `GROQ_API_KEY`, `CEREBRAS_API_KEY`, `OPENROUTER_API_KEY`) and the observability set (`LANGFUSE_PUBLIC_KEY` / `LANGFUSE_SECRET_KEY` plus random-hex secrets for Langfuse’s bundled Postgres). Two rules follow from the deployment history: (i) the gateway reads keys at start-up and *filters out* any provider whose key is absent, so a missing key skips that tier gracefully rather than crashing a request; and (ii) **rotate on exposure** — a write-scoped token leaked during the abandoned HuggingFace attempts, and the lesson is to treat any credential that has touched a transcript, a log, or a pushed repository as compromised and reissue it. No secret is reproduced in this document for the same reason.

47.3 Operational runbook: failure modes and recovery

Every layer degrades rather than crashes (§25); the table below is the recovery reference.

Failure	What the system does	Operator recovery
All four LLM tiers exhausted / down	Fallback chain tries each in order, then the request errors; evaluation records it as a hard failure (§40).	Wait for the daily quota to re-set, or add a paid tier (§41).
Scanned PDF with no text layer	<code>pdf_loader.py</code> auto-renders at 2× and runs Tesseract (<code>ind</code>); a quality flag is recorded if OCR is weak.	None usually (unattended); if OCR text is poor, re-scan at higher resolution and re-drop.
Bad document in the inbox	The watcher quarantines the pair to <code>data/raw/failed/</code> with a reason; a lock file prevents concurrent runs (stale lock auto-clears after ~1 h).	Read the reason, fix the PDF or its <code>.meta.json</code> , re-drop into <code>inbox/</code> .
Vector store or BM25 index corrupt / out of sync	Retrieval returns wrong or empty results.	Rebuild from the parsed JSON, which is the source of truth: <code>python -m app.retrieval.indexer --rebuild</code> . The index is fully derivable.
Registry write interrupted (crash)	Writes are atomic (temp file then rename), so the previous <code>documents.json</code> is intact.	None; re-run the ingest.
A validator throws on a response	The pipeline marks that check unknown rather than crashing; the answer is still returned but flagged for review.	Inspect the HITL queue (<code>data/hitl_queue.jsonl</code>).
Slow first query (~7–9 s cold start)	The FP32 embedder loads and runs its first inference; warm queries are ~1.5–2.5 s (§26).	Pre-warm with one dummy encode at start-up; CLI invocations always pay the cold cost.
Duplicate chunk ids across documents	An extraction artefact (e.g. a mangled chapter heading); the indexer de-duplicates last-write-wins.	Benign; inspect only if a specific document retrieves oddly.

Backup. Because every store is a local file, a backup is a copy of `data/` (PDFs, parsed JSON, Chroma, BM25, registry, history) and `models/` (the ONNX embedder and Tesseract language data). The retired deployment plan scheduled a Chroma-backup cron; locally, a periodic folder copy is sufficient, and even a total loss of the indexes is recoverable from `data/parsed/` via an `indexer --rebuild`.

48. Current Corpus Inventory (2026-05-28)

22 documents — 19 article-structured regulations and 3 reference documents — ~1,800 child chunks at dimension 1024. UU 6/2023 (omnibus) is excluded. Article counts are post-fix (Penjelasan stripped, pre-BAB recovered).

Document	Type	Articles	Subject
permensos-8-2023	PERMENSOS	34	Trafficking & problematic migrant workers (anchor)
permensos-8-2021	PERMENSOS	32	Collection of money/goods
permensos-8-2024	PERMENSOS	5	Collection of money/goods (update)
permensos-5-2024	PERMENSOS	65	Social Welfare Institutions
permensos-21-2019	PERMENSOS	28	(Permensos 21/2019)
permensos-22-2016	PERMENSOS	4	National Standards for Welfare Institutions
permensos-28-2017	PERMENSOS	10	Verification/validation of integrated data
permensos-3-2025	PERMENSOS	29	(Permensos 3/2025)
uu-39-1999	UU	106	Human Rights
uu-35-2014	UU	54	Child Protection (amendment)
uu-20-2023	UU	71	State Civil Apparatus
uu-13-2011	UU	44	Handling of the Poor
uu-9-1961	UU	9	Collection of money/goods (OCR-recovered)
pp-39-2012	PP	78	Implementation of social-welfare programmes
perbup-46-2016	PERBUP	22	Social-affairs organisation, Kediri
perbup-24-2021	PERBUP	12	(Perbup 24/2021)
perbup-39-2022	PERBUP	18	Social-affairs technical-unit organisation
permenkes-21-2020	PERMENKES	10	Health-ministry strategic plan 2020–24
other-7-2023	OTHER	20	Civil-service-agency regulation
rpjmn-2025-2029	reference	—	National Mid-Term Development Plan summary
sop-pendataan-kemiskinan	reference	—	SOP: poverty-data verification
sop-administrasi-surat-keluar	reference	—	SOP: outgoing-mail administration

Deliberately not ingested (low value / high cost): two large statistics books (mostly tables, which chunk poorly) and a 108-page pure-scan SOP (expensive OCR for tangential content). Any can be added later via the generic ingester.

49. Disclaimer

Patih is a regulation-research assistant, not a substitute for a lawyer or a judge. Its accuracy is a calibrated target with human review, not a guarantee. Always verify a cited article against the original regulation before relying on it for any consequential decision.

50. Annotated Bibliography

Every source below shaped a concrete decision; for each we give the citation, the headline claim, and how Patih used it.

50.1 Retrieval architecture

Butler & Butler (2026), *Legal RAG Bench* (arXiv:2603.01710).

Embedder swap gives +17.5 pp correctness vs. a moderate LLM-swap effect; many “hallucinations” are retrieval failures. **Used:** the retrieval-is-the-ceiling principle (§2.4).

Liu et al. (2023), *Lost in the Middle* (arXiv:2307.03172).

U-shaped recall in long contexts. **Used:** why Patih chunks instead of stuffing (§2, §14).

Leng et al. (2024), *Long Context RAG Performance* (arXiv:2411.03538).

Only SOTA models sustain accuracy above 64k tokens. **Used:** empirical companion to the above.

Reuter et al. (2025), *Reliable Retrieval for Large Legal Datasets* (arXiv:2510.06999).

Names DRM and proposes SAC; generic summaries beat expert ones. **Used:** §14; SAC on the roadmap.

Yu & Tsai (2025), *Tree-Based Text Retrieval* (arXiv:2506.13607).

Hierarchical-clustering retrieval without a fixed k . **Used:** nearest analog of Pasal-ayat-huruf chunking (§9).

Cormack et al. (2009), *Reciprocal Rank Fusion* (SIGIR’09).

RRF with $k = 60$. **Used:** the fusion in hybrid.py (§8).

Kalra et al. (2025), *HyPA-RAG* (arXiv:2409.09046).

Query-complexity classifier + adaptive parameters + dense/sparse/KG hybrid. **Used:** Phase 2 reference design.

50.2 Embedder and language-model selection

LazarusNLP (2025), *Indonesian Sentence Embeddings benchmark*.

multilingual-e5-large leads MIRACL-id. **Used:** the embedder choice (§4).

Chen et al. (2024), *BGE-M3* (arXiv:2402.03216).

Unified dense/sparse/multi-vector. **Used:** considered and rejected for Phase 1 simplicity.

Koto et al. (2024), *Cracking the Code* (arXiv:2409.08564).

Small Indonesian models near-random on legal reasoning. **Used:** why Patih does not default to small ID models (§15).

Susanto et al. (2025), *SEA-HELM*.

Qwen 3 leads the SEA leaderboard. **Used:** Qwen on Cerebras as a fallback tier.

50.3 Hallucination defence and citation discipline

Magesh et al. (2024), *Hallucination-Free?* (Stanford HAI).

Commercial legal tools hallucinate 17–34%. **Used:** the realistic accuracy ceiling (§2, §19).

Noël et al. (2025), *HalluGraph* (arXiv:2512.01659).

EG + RP, AUC 0.979 on legal docs. **Used:** directly inspired `entity_grounding.py` and `relation_preservation.py` (§19).

Qian et al. (2024), *Capacity of Citation Generation* (arXiv:2410.11217).

Metrics over-penalise extra citations; generate-then-refine. **Used:** Patih separates precision and recall (§18).

Qian et al. (2025), *VeriCite* (arXiv:2510.11394).

NLI claim verification. **Used:** Patih’s Layer 2 is the regex-and-set variant; NLI is Phase 3.

Zhang et al. (2025), *Measuring Faithfulness and Abstention* (arXiv:2506.00694).

Abstention is a distinct metric most LLMs fail. **Used:** refusal precision as a first-class metric (§24).

Sardana (2025), *Real-Time Evaluation Models for RAG* (arXiv:2503.21157).

No single judge wins; ensembles are better. **Used:** Patih’s four validators are a heterogeneous ensemble.

Liu et al. (2026), *Fast and Faithful* (arXiv:2603.23508).

Real-time verifier with adaptive inference. **Used:** Phase 3 candidate to replace regex EG/RP.

50.4 Agentic and GraphRAG (Phase 2 / 3)

Asai et al. (2023), *Self-RAG* (arXiv:2310.11511).

Retrieve/critique tokens. **Used:** Phase 3 candidate (§21).

Yan et al. (2024), *Corrective RAG*; Yalavarthi (2026) reproduction.

Retrieval-evaluator gating; works on entity alignment. **Used:** Phase 3; reinforces EG.

Jiao et al. (2026), *Doctor-RAG* (arXiv:2604.00865).

Failure-aware repair. **Used:** Phase 3 failure-aware retries.

Wang et al. (2025), *Chain-of-Retrieval* (arXiv:2501.14342).

Dynamic query reformulation. **Used:** Phase 3 multi-hop.

Mishra et al. (2026), *SoK: Agentic RAG* (arXiv:2603.07379).

POMDP formalisation; compounding-hallucination risks. **Used:** the explicit reason for no agentic loop in Phase 1 (§21).

Edge et al. (2024), *From Local to Global: GraphRAG* (arXiv:2404.16130).

KG + community detection + map-reduce QA. **Used:** Phase 2 thematic pattern (§22).

Goel et al. (2026), *Domain-Partitioned Hybrid RAG for Legal Reasoning* (arXiv:2602.23371).

RAG modules + Neo4j KG + orchestrator; 70% vs. 37.5%. **Used:** Phase 2 architectural template.

50.5 Evaluation, observability, production, cross-lingual

Es et al. (2023), *RAGAS* (arXiv:2309.15217).

Reference-free metrics. **Used:** the framework behind app/eval/ (§23).

Pan et al. (2023), *Survey of Vector DBMS* (arXiv:2310.14021).

VDBMS taxonomy. **Used:** Chroma now, Qdrant/pgvector later (§25).

Amanbayev et al. (2026), *Filtered ANN Search*.

Pre- vs. post-filter trade-offs. **Used:** Phase 2 per-regulation filtered ANN.

Badenes-Olmedo et al. (2019), *Cross-language legal retrieval*.

Synset hierarchies. **Used:** “translate the query, not the corpus” (§17).

Qi et al. (2026), *CroSearch-R1* (arXiv:2604.25182).

Cross-lingual RL. **Used:** Phase 3 cross-lingual upgrade.

50.6 Background and adjacent (consulted, not load-bearing today)

- Paul et al. (2023), *Pre-trained LMs for the Legal Domain (Indian Law)*.
- Joshi et al. (2024), *IL-TUR: Indian Legal Text Understanding*.
- Öztürk et al. (2026), *HUKUKBERT (Turkish Law)*.
- Baba Ahmadi et al. (2026), *LEMUR: Multilingual Law Embedding*.
- Han et al. (2025), *RAG with Graphs (GraphRAG survey)*; Ni et al. (2025), *StepChain GraphRAG*; Fan et al. (2026), *Do We Still Need GraphRAG?*.
- Keisha et al. (2025), *Adaptive RAG for Legal Research*; Shanivendra (2026), *Citation-Enforced RAG*; Lelong et al. (2026), *Agentic RAG with KGs*.
- Ockerman et al. (2025), *Distributed Vector Databases on HPC*.